Numerical Analysis of Mean-Field BSDEs:

Deep Learning Solutions for Interacting Agents in Limit Order Books

Study Draft — expanded derivations and full explanations

Christian Garry
Durham University

April 21, 2026

Contents

1 Introduction **4**
1.1 The market-maker's problem . . . 4
1.2 Why BSDEs? . . . 5
1.3 Contributions . . . 5
1.4 Reader's guide . . . 6

2 Mathematical Framework **7**
2.1 Probability-space prerequisites . . . 7
2.2 Forward SDEs and jump-diffusions . . . 8
2.2.1 Itô's formula for jump-diffusions . . . 8
2.3 Backward stochastic differential equations . . . 9
2.3.1 Motivation: going backward with randomness . . . 9
2.3.2 Martingale representation: where Z comes from . . . 9
2.3.3 Existence and uniqueness: Pardoux–Peng . . . 10
2.3.4 A linear example and its Feynman–Kac interpretation . . . 11
2.4 Nonlinear Feynman–Kac . . . 11
2.5 BSDEs with jumps (BSDEJ) . . . 12
2.5.1 Martingale representation with jumps . . . 12
2.6 Mean-field games and McKean–Vlasov equations . . . 13
2.6.1 Interacting particle systems . . . 13
2.6.2 The mean-field limit and propagation of chaos . . . 13
2.6.3 Mean-field games versus mean-field control . . . 14
2.6.4 McKean–Vlasov BSDEs and FBSDEs . . . 14
2.7 The Cont–Xiong dealer market model . . . 14
2.7.1 Primitives . . . 14
2.7.2 Objective . . . 15
2.7.3 Mean-field limit of the CX model . . . 15
2.7.4 HJB equation and quote derivation . . . 16
2.7.5 The CX model as a BSDEJ . . . 17
2.7.6 Forward SDE approximations . . . 18
2.8 Summary . . . 18

3 Deep BSDE Methods **19**
3.1 The deep BSDE paradigm . . . 19
3.1.1 Core insight . . . 19
3.1.2 Convergence results . . . 20
3.1.3 The BatchNorm discovery . . . 20
3.2 Extension to McKean–Vlasov FBSDEs . . . 20
3.2.1 The fictitious-play decomposition . . . 21

3.3 Extension to jump processes (BSDEJ solvers) . . . 21
3.3.1 Naive per-timestep BSDEJ solver . . . 21
3.3.2 Shared-weights + warm-start BSDEJ solver . . . 22
3.3.3 Compensated martingale ablation . . . 22
3.4 Architectural failure modes of MV deep BSDE solvers . . . 23
3.4.1 Mean-field conditioning . . . 23
3.4.2 Failure 1: generator bypass . . . 23
3.4.3 Failure 2: BatchNorm erasure of broadcast features . . . 23
3.4.4 Failure 3: representation collapse under DeepSets + symmetric inputs 25
3.4.5 Corrected architecture: putting the fixes together . . . 26
3.4.6 FiLM conditioning as an alternative . . . 26
3.5 Summary . . . 26

4 The Neural Bellman Solver 27
4.1 Stationary formulation . . . 27
4.1.1 Neural fictitious play: coupling to the population . . . 28
4.2 Validation against the exact solver . . . 28
4.2.1 Benchmark setup . . . 28
4.2.2 Results: agreement with exact solver . . . 29
4.2.3 Q-scaling: where the tabular method stalls . . . 29
4.2.4 N-scaling: mean-field regime confirmed . . . 29
4.3 Extension to continuous inventory . . . 30
4.3.1 From FOC to continuous quotes . . . 30
4.3.2 BSDE formulation . . . 30
4.3.3 Validation . . . 30
4.4 Finite-horizon BSDEJ solver . . . 30
4.4.1 Architecture . . . 31
4.4.2 Warm-start from fictitious play . . . 31
4.4.3 Validation . . . 31
4.5 Summary of validated numerical results . . . 31

5 Nash Equilibrium and Tacit Collusion 33
5.1 Equilibrium concepts . . . 33
5.1.1 Nash equilibrium . . . 33
5.1.2 Pareto optimum (cartel) . . . 34
5.1.3 Nash vs Pareto: the prisoner's dilemma structure . . . 34
5.1.4 Exact benchmarks from the committed JSONs . . . 34
5.2 Multi-agent deep reinforcement learning . . . 34
5.2.1 DDPG for single-agent continuous control . . . 35
5.2.2 MADDPG: one actor-critic per agent . . . 35
5.2.3 Reward function . . . 35
5.2.4 Actor and critic architectures . . . 36
5.2.5 Pretraining from the monopolist solution . . . 36
5.2.6 The cluster experiment . . . 36
5.3 Tacit collusion: the main result . . . 36
5.3.1 What "tacit collusion" means . . . 36
5.3.2 Mechanisms producing tacit collusion . . . 37
5.3.3 Statistical significance . . . 37
5.3.4 Regulatory implications . . . 38
5.4 Summary . . . 38

6 Conclusion **39**
6.1 Summary of contributions . . . 39
6.2 Relationship to prior work . . . 40
6.3 Limitations . . . 40
6.4 Future work . . . 41
6.5 Closing remarks . . . 42

A Proof sketches **43**
A.1 Existence of BSDE solutions: full Pardoux–Peng argument . . . 43
A.2 Deep BSDE consistency: Beck–E–Jentzen theorem . . . 45
A.3 Propagation of chaos: Sznitman's coupling . . . 45

B Implementation details **46**
B.1 Code pointers by chapter . . . 46
B.2 Typical hyperparameters . . . 46
B.3 Reproducibility notes . . . 47
B.4 Hardware and runtime notes . . . 47
B.5 Dependencies . . . 47

C Notation glossary **48**

Chapter 1

Introduction

1.1 The market-maker's problem

A *market maker* is a specialist trader whose role is to provide liquidity: they stand ready to both buy and sell a security, earning the bid–ask spread in compensation for the risks they bear. In a modern electronic exchange, they do this by posting limit orders in the *limit order book* (LOB): a queue of outstanding buy and sell offers at different prices.

Two numbers describe their quoting strategy at any instant:

- $\delta_t^a \geq 0$: the half-spread on the ask side — the distance from the mid-price at which they are willing to sell.
- $\delta_t^b \geq 0$: the half-spread on the bid side — the distance at which they are willing to buy.

When an incoming market order arrives, it consumes whichever side is best quoted; the market maker's inventory q_t jumps up or down by one unit, and they earn δ_t^a or δ_t^b.

Three intertwined forces shape the optimal (δ^a, δ^b):

1. **The fill-rate / margin trade-off.** The further you quote from the mid, the less likely you are to be hit, but the more you earn per fill. In the workhorse Avellaneda–Stoikov (2008) [**?**] / Cont–Xiong (2024) [**?**] setup, fill intensities are exponential: $\lambda(\delta) \propto e^{-\alpha\delta}$. A monopolist facing no competition quotes at the expected-revenue maximum $\delta^* = 1/\alpha$.
2. **Inventory risk.** You do not want to end up heavily long or short, because price moves in the "wrong" direction hurt you. A quadratic penalty $\psi(q) = \phi q^2$ on running and terminal inventory pushes you to skew quotes: tighter ask when long, tighter bid when short, so that fills naturally *mean-revert* inventory.
3. **Competition.** Other market makers are quoting the same asset. Your effective fill rate depends on the distribution of *their* quotes. If everyone tightens, your margin erodes; if everyone widens, you can too. This mean-field coupling turns a single-agent control problem into a fixed-point / equilibrium problem.

These three forces interact nontrivially. Cont and Xiong [**?**] showed that the fully dynamic equilibrium admits a rich set of outcomes: a *Nash equilibrium* where each agent best-responds to the others, a *Pareto / cartel* solution where agents coordinate on wider spreads, and — under learning dynamics — a *tacit-collusion* regime where self-interested RL agents converge strictly above Nash without explicit coordination. Explaining, validating and quantifying these outcomes at arbitrary N is the central motivating question of this dissertation.

1.2 Why BSDEs?

The classical tool for stochastic optimal control is the Hamilton–Jacobi–Bellman (HJB) equation: the value function V satisfies a nonlinear second-order PDE whose supremum over controls defines the optimal strategy. In low dimensions one solves HJB directly on a grid. In high dimensions the grid method collapses under the curse of dimensionality.

An equivalent formulation — and the one we exploit — is the *backward stochastic differential equation* (BSDE):

$$-\mathrm{d}Y_t = f(t, X_t, Y_t, Z_t)\,\mathrm{d}t - Z_t \mathrm{d}W_t, \qquad Y_T = g(X_T). \tag{1.1}$$

Discovered by Pardoux and Peng (1990) [**?**], BSDEs are the Markovian "backward in time" cousin of forward SDEs. Their value Y_t equals the value function $V(t, X_t)$ along the state trajectory, and the process Z_t equals the (scaled) gradient $\sigma^\top \nabla V$.

For our market-making problem, the forward process is driven by Poisson jump events (fills), not diffusions. The natural generalisation is the BSDE with jumps — the *BSDEJ* of Tang and Li (1994) [**?**] — in which Z_t is supplemented by a jump integrand $U_t(e)$ encoding the post-jump value change. We show in Chapter 2 that the Cont–Xiong dealer market is equivalent to a specific BSDEJ whose generator and terminal we will derive from first principles.

Two advantages motivate this route:

- **Dimension.** BSDEJs can be solved via Monte Carlo simulation whose cost scales linearly in the state dimension, whereas grid-based PDE methods scale exponentially. For high-dimensional extensions (multi-asset, common noise, heterogeneous agents) this is the only practical route.
- **Nonlinearity.** The HJB generator f is naturally nonlinear in (Y, Z, U) because the supremum over controls produces a Hamiltonian. BSDE theory accommodates arbitrary Lipschitz nonlinearities.

The modern tool for actually solving BSDEs in practice is the *deep BSDE* method of Han, Jentzen and E (2017/2018) [**?**], in which Z_t (and, in our jump version, U_t) is parameterised by a neural network and trained via backpropagation through the discretised BSDE trajectory. We describe this method in detail in Chapter 3 and devote substantial attention to the *architectural conditions* under which the method works — conditions we found the hard way (Chapter 3, §3).

1.3 Contributions

This dissertation makes four contributions:

(C1) **A validated neural Bellman solver for the stationary Cont–Xiong model.** We implement and validate a deep BSDE solver against the exact tabular fictitious-play solution of Cont–Xiong (their Algorithm 1). Across $N \in \{2, 5, 10\}$ agents we achieve spread errors below 1% and value-function errors below 2%. The method's main advantage is that it scales to regimes (large Q, continuous inventory, 3-D state with adverse selection) where Algorithm 1 becomes intractable. Chapter 4.

(C2) **Architectural diagnostics for McKean–Vlasov deep BSDEs.** In companion preprint [**?**] (included here as Chapter 3 §3) we identify three independent failure modes that silently suppress the mean-field signal in deep BSDE solvers:

- *Generator bypass*: the BSDE generator did not use the law embedding, so the optimiser had no gradient signal from the loss back to the encoder.
- *BatchNorm erasure*: $\Phi(\mu_t)$ is broadcast-identical across the batch, forcing zero batch variance and causing BatchNorm to zero out the feature.
- *Representation collapse*: under symmetric inputs, a random DeepSets encoder cancels variance information in its mean-pooling step.

After correcting all three, the learned competitive factor $h(\Phi(\mu_t))$ varies by $4\times$ across population shapes, and optimal quotes shift by $2\times$, with quote skew flipping sign — empirically validating mean-field effects that the architecture had previously masked.

(C3) **A finite-horizon BSDEJ solver** for the jump form of the Cont–Xiong model (§3.3). Unlike the diffusion-approximation solver, this version learns the jump adjoints U^a, U^b directly and does not require the moment-matched Gaussian reduction. We show a version using shared weights across time steps plus a warm-start initialisation achieves 2.6% error against the exact benchmark (Chapter 4).

(C4) **Statistical evidence of tacit collusion in MADDPG-trained dealers.** In Chapter 5 we train decentralised actor–critic agents on the CX market using the MADDPG algorithm. At $N = 2$, 15 of 20 independent Hamilton-cluster seeds converge to spreads strictly above the single-shot Nash value ($p = 0.002$, one-sided binomial test); at $N = 5$, 5 of 5 seeds do. The outcome is not explicitly coordinated — it emerges from the combination of monopolist pretraining and slow gradient dynamics.

1.4 Reader's guide

The dissertation can be read linearly; each chapter assumes the previous. Readers already fluent in a topic can jump in:

- Chapter 2 builds the mathematical scaffolding. If you know BSDEs, BSDEJs, the nonlinear Feynman–Kac theorem, and mean-field games, you can skim this, though §2.7.5 (the CX model $\leftrightarrow$ BSDEJ identification) and §2.7.3 (the mean-field limit under discrete jumps) are non-standard.
- Chapter 3 covers the deep BSDE methodology. If you know Han–Jentzen–E (2018) and Han–Hu–Long (2022), skip directly to §3.4 for the architectural findings.
- Chapters 4 and 5 are the empirical core. Chapter 4 is the main computational result (a validated solver). Chapter 5 is the main economic result (tacit collusion under learning dynamics).
- Chapter 6 summarises, acknowledges limitations, and proposes extensions.
- Appendix A collects proof sketches; Appendix B collects implementation and reproducibility details.

Throughout, I adopt the pedagogical convention of pairing every non-trivial theorem with a *plain-English* paragraph before (giving intuition) and after (connecting the formal statement back to the application). A "study-draft" like this is meant to be read, not scanned; the additional prose trades page-count for clarity of ideas. Where relevant, I cite the specific repo file and line that implements a given equation.

Chapter 2

Mathematical Framework

This chapter develops the mathematical machinery that every subsequent chapter relies on: stochastic calculus with jumps, backward stochastic differential equations (BSDEs and BSDEJs), the nonlinear Feynman–Kac correspondence between BSDEs and semilinear PDEs, mean-field games and the McKean–Vlasov formulation, and finally the Cont–Xiong dealer market model whose reformulation as a BSDEJ justifies everything we do afterwards.

The style is explanatory: every non-trivial theorem is preceded by a motivating paragraph and followed by consequences or caveats. Derivations are spelled out step by step; where a clever manipulation appears, the paragraph before the equation explains what we are about to do and why.

2.1 Probability-space prerequisites

We fix a filtered probability space $(\Omega, \mathcal{F}, (\mathcal{F}_t)_{t\geq 0}, \mathbb{P})$ on which all our stochastic processes live. The filtration $(\mathcal{F}_t)_{t\geq 0}$ represents the information available at time t; it is *right-continuous* ($\mathcal{F}_t = \bigcap_{s>t} \mathcal{F}_s$) and *complete* ($\mathcal{F}_0$ contains all $\mathbb{P}$-null sets of $\mathcal{F}$). These technical conditions — together known as the *usual conditions* — are what make Itô calculus work without pathologies.

A d-dimensional *standard Brownian motion* $W = (W_t)_{t\geq 0}$ is a process with the three defining properties:

1. $W_0 = 0$ almost surely.
2. For $0 \leq s < t$ the increment $W_t - W_s$ is $\mathcal{N}(0, (t-s)I_d)$-distributed and independent of $\mathcal{F}_s$.
3. The paths $t \mapsto W_t$ are continuous almost surely.

The Itô integral $\int_0^t H_s \, \mathrm{d}W_s$ is defined for adapted, square-integrable integrands H by first defining it on simple processes and extending by L^2-isometry (Karatzas–Shreve [**?**], Ch. 3). The key facts we will use repeatedly are:

- Itô integrals are martingales: $\mathbb{E}\big[\int_0^t H_s \mathrm{d}W_s \;\big|\; \mathcal{F}_u\big] = \int_0^u H_s \mathrm{d}W_s$ for $u < t$.
- The Itô isometry: $\mathbb{E}\big[(\int_0^t H_s \mathrm{d}W_s)^2\big] = \int_0^t \mathbb{E}[H_s^2] \, \mathrm{d}s$.
- The quadratic-variation identity: $(\mathrm{d}W_t)^2 = \mathrm{d}t$ in the sense of the Itô multiplication table $(\mathrm{d}t)^2 = \mathrm{d}t \, \mathrm{d}W_t = 0$, $(\mathrm{d}W_t)^2 = \mathrm{d}t$.

A *Poisson random measure* N on $[0,T] \times E$ with intensity measure $\nu(\mathrm{d}e)\,\mathrm{d}t$ is defined by: for disjoint Borel sets $A \subset E$ and time intervals $[s,t]$, $N([s,t],A)$ is Poisson-distributed with mean $(t-s)\nu(A)$, and counts over disjoint regions are independent. The *compensated* random measure $\tilde{N}(\mathrm{d}t,\mathrm{d}e) := N(\mathrm{d}t,\mathrm{d}e) - \nu(\mathrm{d}e)\,\mathrm{d}t$ is a martingale measure: for $\phi : E \to \mathbb{R}$ with $\int_E \phi^2 \,\mathrm{d}\nu < \infty$, the stochastic integral

$$M_t := \int_0^t \int_E \phi(e)\,\tilde{N}(\mathrm{d}s,\mathrm{d}e) \tag{2.1}$$

is a martingale with quadratic variation $\langle M \rangle_t = t \int_E \phi^2 \,\mathrm{d}\nu$.

Intuition. Think of N as a rain of random dots on the (t,e) plane with density $\nu(\mathrm{d}e)\,\mathrm{d}t$. The integral $\int \phi(e)\,N(\mathrm{d}t,\mathrm{d}e)$ is the sum of ϕ evaluated at each dot. Compensation subtracts the expected rate so the result is mean-zero. In our dealer market, the "dots" will be execution events, with mark $e = \pm 1$ indicating which side filled.

2.2 Forward SDEs and jump-diffusions

A *jump-diffusion* is a process $X = (X_t)_{t \geq 0}$ satisfying

$$\mathrm{d}X_t = \mu(t,X_t)\,\mathrm{d}t + \sigma(t,X_t)\,\mathrm{d}W_t + \int_E \gamma(t,X_{t-},e)\,\tilde{N}(\mathrm{d}t,\mathrm{d}e), \tag{2.2}$$

with initial condition $X_0 = x_0 \in \mathbb{R}^n$. The coefficients $\mu : [0,T] \times \mathbb{R}^n \to \mathbb{R}^n$, $\sigma : [0,T] \times \mathbb{R}^n \to \mathbb{R}^{n \times d}$, $\gamma : [0,T] \times \mathbb{R}^n \times E \to \mathbb{R}^n$ are measurable. The notation X_{t-} denotes the left limit, reflecting that the jump coefficient is evaluated *just before* the jump event.

Under Lipschitz conditions on μ, σ, γ (uniform in t, e; for γ, $L^2(\nu)$-Lipschitz), Equation (2.2) admits a unique càdlàg adapted solution with $\mathbb{E}[\sup_{t \leq T} \|X_t\|^2] < \infty$ (Applebaum [?], Ch. 6).

2.2.1 Itô's formula for jump-diffusions

The workhorse of all subsequent derivations is the generalisation of Itô's formula to jump-diffusions. Let $f \in C^{1,2}([0,T] \times \mathbb{R}^n)$ — once continuously differentiable in t and twice in x. Along the trajectory X_t,

$$\begin{aligned} \mathrm{d}f(t,X_t) = \partial_t f\,\mathrm{d}t \;+\; \nabla f^\top \mu\,\mathrm{d}t + \nabla f^\top \sigma\,\mathrm{d}W_t \;+\; \tfrac{1}{2}\operatorname{Tr}\big(\sigma\sigma^\top \nabla^2 f\big)\,\mathrm{d}t \\ + \int_E \big[f(t,X_{t-} + \gamma(t,X_{t-},e)) - f(t,X_{t-})\big]\,\tilde{N}(\mathrm{d}t,\mathrm{d}e) \\ + \int_E \big[f(t,X_{t-} + \gamma) - f(t,X_{t-}) - \gamma^\top \nabla f\big]\,\nu(\mathrm{d}e)\,\mathrm{d}t, \end{aligned} \tag{2.3}$$

where all derivatives of f are evaluated at (t,X_{t-}). Three structural remarks deserve emphasis:

1. The *continuous part* of $\mathrm{d}f$ consists of the drift $\mu^\top \nabla f$, the diffusion martingale $\sigma^\top \nabla f\,\mathrm{d}W_t$, and the Itô correction $\frac{1}{2}\operatorname{Tr}(\sigma\sigma^\top \nabla^2 f)\,\mathrm{d}t$. Without jumps, this is the classical Itô formula.
2. The *jump martingale part* $\int_E [f(t,X_{t-}+\gamma) - f(t,X_{t-})]\tilde{N}(\mathrm{d}t,\mathrm{d}e)$ accumulates the value changes at each jump, compensated by the intensity.
3. The *compensator drift* $\int_E [f(\cdot + \gamma) - f - \gamma^\top \nabla f]\,\nu(\mathrm{d}e)\,\mathrm{d}t$ is the difference between the true post-jump value of f and its first-order (linear) approximation. This is what makes the jump martingale term have zero mean; without it we would double-count.

Intuition. For a diffusion, the Itô correction $\frac{1}{2}f''\sigma^2\,\mathrm{d}t$ arises because $(\mathrm{d}W)^2 = \mathrm{d}t$: the second-order Taylor term does not vanish in the small-time limit. For jumps, the story is different. Jumps are $O(1)$ in size, not infinitesimal, so Taylor expansion would be wrong. Instead we evaluate f at the post-jump state and subtract off the pre-jump value directly; the compensator drift is the deterministic piece that makes the martingale part genuinely mean-zero.

2.3 Backward stochastic differential equations

2.3.1 Motivation: going backward with randomness

Suppose you know a random terminal value $\xi = g(X_T)$ and you want to compute its value "now". In a deterministic world you would just solve an ODE backwards. In a stochastic world the naive backward ODE is ill-posed: the solution at time t has to be $\mathcal{F}_t$-measurable (adapted to the information available at time t), yet the terminal condition involves future randomness. Running a deterministic ODE backwards from $g(X_T)$ would produce a value that depends on information from the future — a violation of adaptedness.

Pardoux and Peng (1990) [**?**] resolved this by introducing a second, compensating process. A BSDE is a pair (Y, Z) of adapted processes satisfying

$$Y_t = g(X_T) + \int_t^T f(s, X_s, Y_s, Z_s)\,\mathrm{d}s - \int_t^T Z_s\,\mathrm{d}W_s, \tag{2.4}$$

or, in differential form,

$$-\mathrm{d}Y_t = f(t, X_t, Y_t, Z_t)\,\mathrm{d}t - Z_t\,\mathrm{d}W_t, \qquad Y_T = g(X_T). \tag{2.5}$$

Here $f : [0,T] \times \mathbb{R}^n \times \mathbb{R} \times \mathbb{R}^d \to \mathbb{R}$ is the *generator* (or driver), and $g : \mathbb{R}^n \to \mathbb{R}$ is the *terminal condition.* Y is scalar-valued, Z is d-dimensional (matching the Brownian motion W).

The key insight is that the extra process Z_t absorbs the randomness that would otherwise violate adaptedness: if we subtract $\int_t^T Z_s\,\mathrm{d}W_s$ from the backward integral, we get an adapted process on the left. The next proposition makes this precise.

2.3.2 Martingale representation: where Z comes from

Theorem 2.1 (Martingale representation in the Brownian filtration)**.** *Let $(\mathcal{F}_t)_{t\geq 0}$ be the filtration generated by W, completed by $\mathbb{P}$-null sets. For every square-integrable random variable $\xi \in L^2(\Omega, \mathcal{F}_T, \mathbb{P})$, there exists a unique (up to $\mathrm{d}t\otimes\mathrm{d}\mathbb{P}$-null sets) adapted process $Z = (Z_t)_{t\in[0,T]}$ with $\mathbb{E}\int_0^T \|Z_s\|^2\,\mathrm{d}s < \infty$ such that*

$$\xi = \mathbb{E}[\xi] + \int_0^T Z_s\,\mathrm{d}W_s. \tag{2.6}$$

Consequently, the martingale $M_t := \mathbb{E}[\xi \mid \mathcal{F}_t]$ is a stochastic integral:

$$M_t = \mathbb{E}[\xi] + \int_0^t Z_s\,\mathrm{d}W_s. \tag{2.7}$$

Intuition. Every square-integrable $\mathcal{F}_t$-measurable random variable can be written as its expectation plus an Itô integral of some adapted Z. In other words, in the Brownian filtration *all martingales are stochastic integrals.* This is what makes BSDEs well-posed: given $g(X_T)$, the process Z in (2.4) is forced on us by the theorem.

Proof idea. Polynomials in $W_{t_1}, \dots, W_{t_k}$ are dense in $L^2(\Omega, \mathcal{F}_T, \mathbb{P})$, and for each such polynomial the representation can be written explicitly (by iterated Itô integrals). The claim follows by a density argument. See Karatzas–Shreve [?] Thm. 3.4.15 or El Karoui et al. [?] §2. □

2.3.3 Existence and uniqueness: Pardoux–Peng

The price we pay for the backward structure is non-linearity: the generator f in (2.4) depends on (Y_s, Z_s) in general. Nevertheless, under Lipschitz conditions the BSDE admits a unique solution.

Assumption 2.2 (Standard Lipschitz)**.** The generator $f : [0,T] \times \mathbb{R}^n \times \mathbb{R} \times \mathbb{R}^d \to \mathbb{R}$ is measurable, and there exists $K > 0$ such that for all t, x, y_1, y_2, z_1, z_2,

$$|f(t,x,y_1,z_1) - f(t,x,y_2,z_2)| \le K\big(|y_1 - y_2| + \|z_1 - z_2\|\big). \tag{2.8}$$

Moreover $\mathbb{E}\int_0^T |f(t, X_t, 0, 0)|^2 \,\mathrm{d}t < \infty$ and $\mathbb{E}[g(X_T)^2] < \infty$.

Theorem 2.3 (Pardoux–Peng [?])**.** *Under Assumption 2.2, the BSDE* (2.4) *admits a unique pair* (Y, Z) *of adapted processes with*

$$\mathbb{E}\Big[\sup_{t\in[0,T]} |Y_t|^2\Big] + \mathbb{E}\int_0^T \|Z_s\|^2 \,\mathrm{d}s < \infty. \tag{2.9}$$

Proof sketch (fixed-point argument). The proof is a Banach fixed-point argument in the space $\mathcal{H}^2$ of pairs (Y, Z) with finite norm

$$\|(Y,Z)\|_\beta^2 := \mathbb{E}\int_0^T e^{\beta t}\big(|Y_t|^2 + \|Z_t\|^2\big)\,\mathrm{d}t,$$

for a parameter $\beta > 0$ chosen large enough to make the map below contractive.

Step 1: Defining the map. Given $(y, z) \in \mathcal{H}^2$, define $\xi := g(X_T) + \int_0^T f(s, X_s, y_s, z_s)\,\mathrm{d}s$. By the Lipschitz + growth assumptions, $\xi \in L^2$. Apply the martingale representation theorem to obtain adapted $\tilde{Z}$ with

$$\mathbb{E}[\xi \mid \mathcal{F}_t] = \mathbb{E}[\xi] + \int_0^t \tilde{Z}_s \,\mathrm{d}W_s.$$

Now set

$$Y_t := \mathbb{E}[\xi \mid \mathcal{F}_t] - \int_0^t f(s, X_s, y_s, z_s)\,\mathrm{d}s, \tag{2.10}$$

$$Z_t := \tilde{Z}_t. \tag{2.11}$$

One checks $Y_T = g(X_T)$ and that (Y, Z) satisfies the BSDE (2.4) with generator frozen at (y, z). The map $(y, z) \mapsto (Y, Z)$ is denoted Θ.

Step 2: Θ is a contraction for β large. Take two inputs $(y_1, z_1), (y_2, z_2)$ with outputs $(Y_1, Z_1), (Y_2, Z_2)$. Using Itô's formula applied to $(Y_1 - Y_2)^2 e^{\beta t}$, integrating from 0 to T, taking expectations, and using the Lipschitz bound on f, one arrives at an estimate

$$\|(Y_1 - Y_2, Z_1 - Z_2)\|_\beta^2 \le \frac{C(K)}{\beta}\|(y_1 - y_2, z_1 - z_2)\|_\beta^2,$$

for a constant $C(K)$ depending only on the Lipschitz constant. Choosing $\beta > C(K)$ makes Θ a strict contraction.

Step 3: Banach fixed point. By Banach's fixed-point theorem, Θ has a unique fixed point $(Y, Z) \in \mathcal{H}^2$. This is the desired solution. Uniqueness is inherited from the fixed-point uniqueness. The L^2-bound on $\sup_t |Y_t|$ follows from Doob's inequality applied to the martingale $\mathbb{E}[\xi \mid \mathcal{F}_t] - \mathbb{E}[\xi]$. □

Remark 2.4 (Non-Lipschitz extensions). The Lipschitz assumption can be relaxed. Kobylanski (2000) [?] covered quadratic-in-z generators via exponential transforms. Briand and Hu (2006) [?] handled unbounded terminal conditions. For this dissertation, all generators are Lipschitz on the bounded quote region (enforced via explicit clamping in code), so Pardoux–Peng suffices.

2.3.4 A linear example and its Feynman–Kac interpretation

When the generator is linear in (Y, Z), the BSDE reduces to a conditional expectation. Concretely, take $f(t, X_t, y, z) = -ry$ (just discounting) and $g(X_T)$ arbitrary. The BSDE becomes

$$-\mathrm{d}Y_t = -rY_t\,\mathrm{d}t - Z_t\,\mathrm{d}W_t, \qquad Y_T = g(X_T). \tag{2.12}$$

Define the discounted process $\tilde{Y}_t := e^{-rt}Y_t$. By Itô,

$$\mathrm{d}\tilde{Y}_t = -re^{-rt}Y_t\,\mathrm{d}t + e^{-rt}\mathrm{d}Y_t = e^{-rt}Z_t\,\mathrm{d}W_t, \tag{2.13}$$

so $\tilde{Y}$ is a martingale. Hence $\tilde{Y}_t = \mathbb{E}[\tilde{Y}_T \mid \mathcal{F}_t] = e^{-rT}\mathbb{E}[g(X_T) \mid \mathcal{F}_t]$, and we recover the classical Feynman–Kac formula:

$$Y_t = e^{-r(T-t)}\mathbb{E}[g(X_T) \mid \mathcal{F}_t] = e^{-r(T-t)}\,u(t, X_t), \tag{2.14}$$

where $u(t, x) := \mathbb{E}[g(X_T) \mid X_t = x]$ is the classical solution of the linear PDE $\partial_t u + \mathcal{L}u - ru = 0$ with terminal $u(T, x) = g(x)$ (here $\mathcal{L}$ is the infinitesimal generator of X).

This identification generalises: the nonlinear BSDE (2.4) corresponds to a nonlinear PDE — the generalised (nonlinear) Feynman–Kac theorem, which we prove next.

2.4 Nonlinear Feynman–Kac

Theorem 2.5 (Nonlinear Feynman–Kac, Markovian case)**.** *Suppose X is a jump-diffusion* (2.2) *with infinitesimal generator $\mathcal{L}$, the functions $\mu, \sigma, \gamma, f, g$ are sufficiently regular, and $u \in C^{1,2}([0, T] \times \mathbb{R}^n)$ satisfies the semilinear PIDE*

$$\partial_t u + \mathcal{L}u + f\big(t, x, u, \sigma^\top \nabla u, \mathcal{J}u\big) = 0, \qquad u(T, x) = g(x), \tag{2.15}$$

where

$$\mathcal{L}u = \mu^\top \nabla u + \tfrac{1}{2}\operatorname{Tr}(\sigma\sigma^\top \nabla^2 u) + \int_E \big[u(t, x+\gamma) - u(t, x) - \gamma^\top \nabla u\big]\nu(\mathrm{d}e),$$

and $\mathcal{J}u(t, x, e) := u(t, x + \gamma(t, x, e)) - u(t, x)$ is the jump operator. Then

$$Y_t = u(t, X_t), \qquad Z_t = \sigma(t, X_t)^\top \nabla u(t, X_t), \qquad U_t(e) = \mathcal{J}u(t, X_{t-}, e) \tag{2.16}$$

solve the BSDEJ with generator f and terminal g.

Proof. Apply Itô's formula (2.3) to $u(t, X_t)$:

$$\begin{aligned}\mathrm{d}u(t, X_t) = {} & \big[\partial_t u + \mu^\top \nabla u + \tfrac{1}{2}\operatorname{Tr}(\sigma\sigma^\top \nabla^2 u)\big]\mathrm{d}t + (\sigma^\top \nabla u)^\top \mathrm{d}W_t \\ & + \int_E \mathcal{J}u\,\tilde{N}(\mathrm{d}t, \mathrm{d}e) + \int_E [\mathcal{J}u - \gamma^\top \nabla u]\,\nu(\mathrm{d}e)\,\mathrm{d}t.\end{aligned}$$

The three drift terms collect to $\mathcal{L}u\,\mathrm{d}t$. Using the PIDE (2.15), $\mathcal{L}u = -\partial_t u - f$, but the $\partial_t u\,\mathrm{d}t$ is already on the left side; so after substitution the total drift becomes $-f\,\mathrm{d}t$. Hence

$$\mathrm{d}u(t, X_t) = -f\big(t, X_t, u, \sigma^\top \nabla u, \mathcal{J}u\big)\mathrm{d}t + (\sigma^\top \nabla u)^\top \mathrm{d}W_t + \int_E \mathcal{J}u\,\tilde{N}(\mathrm{d}t, \mathrm{d}e).$$

This is precisely the BSDEJ dynamics for $Y_t = u(t, X_t)$, $Z_t = \sigma^\top \nabla u$, $U_t = \mathcal{J}u$. The terminal condition is inherited from $u(T, \cdot) = g$. □

Intuition. Theorem 2.5 is the bridge between two seemingly distinct objects: a semilinear parabolic PIDE (the HJB of stochastic control) and a BSDEJ (the "backward" dynamics of a conditional expectation plus a nonlinear generator). Whichever representation is more tractable for a given problem is the one you use:

- In low dimensions, solve the PIDE on a grid. This gives u globally, including $\nabla u, \nabla^2 u$ and the jump operator $\mathcal{J}u$.
- In high dimensions, simulate the BSDEJ via Monte Carlo. This gives (Y_t, Z_t, U_t) at sampled points but scales linearly in state dimension.

Deep BSDE methods (Chapter 3) exploit the latter: they parameterise (Z_t, U_t) by neural networks and train to minimise the terminal mismatch. The neural-network parameters are implicitly learning the derivatives of u at sampled points.

2.5 BSDEs with jumps (BSDEJ)

The natural generalisation of a BSDE to jump-diffusions adds a third unknown: the jump integrand U.

Definition 2.6 (BSDEJ)**.** A *backward stochastic differential equation with jumps* is a triple (Y, Z, U) of adapted processes satisfying

$$Y_t = g(X_T) + \int_t^T f(s, X_{s-}, Y_{s-}, Z_s, U_s)\,\mathrm{d}s - \int_t^T Z_s\,\mathrm{d}W_s - \int_t^T \int_E U_s(e)\,\tilde{N}(\mathrm{d}s, \mathrm{d}e), \quad (2.17)$$

where $U_s : E \to \mathbb{R}$ is the jump integrand at time s, in the sense that $U : [0, T] \times \Omega \times E \to \mathbb{R}$ is predictable.

In differential form,

$$-\mathrm{d}Y_t = f(t, X_{t-}, Y_{t-}, Z_t, U_t)\,\mathrm{d}t - Z_t\,\mathrm{d}W_t - \int_E U_t(e)\,\tilde{N}(\mathrm{d}t, \mathrm{d}e), \quad Y_T = g(X_T). \quad (2.18)$$

The role of $U_t(e)$ is to absorb the mean-zero martingale component of Y_t associated with jumps of size e. It is the *jump adjoint*: the change in Y caused by a unit jump of size e.

2.5.1 Martingale representation with jumps

For the BSDEJ to be well-posed, we need a Brownian-plus-jump analogue of Theorem 2.1.

Theorem 2.7 (Jacod–Yor martingale representation [**?**])**.** *In the filtration $(\mathcal{F}_t)_{t\geq 0}$ generated by W and N (both appropriately completed), every square-integrable $\mathcal{F}_T$-measurable random variable ξ admits a unique representation*

$$\xi = \mathbb{E}[\xi] + \int_0^T Z_s\,\mathrm{d}W_s + \int_0^T \int_E U_s(e)\,\tilde{N}(\mathrm{d}s, \mathrm{d}e), \quad (2.19)$$

with Z adapted in $L^2(\mathrm{d}t \otimes \mathbb{P})$ and U predictable in $L^2(\mathrm{d}t \otimes \mathbb{P} \otimes \nu)$.

This is the "two-source" version of martingale representation: both the diffusion and the jump martingale measure contribute integrands. The BSDEJ existence proof (Tang–Li [?], Barles–Buckdahn–Pardoux [?]) is a fixed-point argument analogous to Pardoux–Peng, with Lipschitz in (y, z, u) replacing Lipschitz in (y, z). We omit the details; the result we use is:

Theorem 2.8 (BSDEJ existence and uniqueness)**.** *Under the analogue of Assumption 2.2 with U included in the Lipschitz bound (measuring $U_1 - U_2$ in the $L^2(\nu)$ norm), the BSDEJ* (2.17) *admits a unique adapted solution* (Y, Z, U).

2.6 Mean-field games and McKean–Vlasov equations

2.6.1 Interacting particle systems

Consider a system of N identical agents, each with state $X_t^{(i)} \in \mathbb{R}^n$, evolving as

$$\mathrm{d}X_t^{(i)} = b\big(X_t^{(i)}, \mu_t^N\big)\,\mathrm{d}t + \sigma\big(X_t^{(i)}, \mu_t^N\big)\,\mathrm{d}W_t^{(i)}, \qquad i = 1, \dots, N, \tag{2.20}$$

where $W^{(i)}$ are independent Brownian motions and

$$\mu_t^N := \frac{1}{N}\sum_{j=1}^{N} \delta_{X_t^{(j)}} \tag{2.21}$$

is the empirical distribution of the whole population. Each agent's drift and diffusion depend on the *state of everyone else* only through the empirical distribution μ_t^N. The dynamics are symmetric — exchangeable — in the sense that permuting the particle labels leaves the law of the system invariant.

2.6.2 The mean-field limit and propagation of chaos

As $N \to \infty$, Sznitman [?] established that for Lipschitz b, σ in (x, μ) (with the Wasserstein-2 metric on μ), the empirical distribution μ_t^N converges to a deterministic limit μ_t, and each agent decouples: in the limit, a representative particle satisfies the *McKean–Vlasov SDE*

$$\mathrm{d}X_t = b(X_t, \mu_t)\,\mathrm{d}t + \sigma(X_t, \mu_t)\,\mathrm{d}W_t, \qquad \mu_t = \mathrm{Law}(X_t). \tag{2.22}$$

The self-referential structure is striking: μ_t is the law of the very process it drives. Existence and uniqueness for (2.22) follow from a Picard iteration in Wasserstein distance (Carmona–Delarue [?], Vol. I §3).

Theorem 2.9 (Propagation of chaos)**.** *Under Lipschitz conditions on b, σ, let $X^{(i)}$ solve* (2.20) *and $X^{\infty,i}$ be i.i.d. copies of the McKean–Vlasov solution* (2.22)*. For any fixed $k \in \mathbb{N}$,*

$$\sup_{t \le T} \mathbb{E}\Big[\big\|X_t^{(1)} - X_t^{\infty,1}\big\|^2\Big] \le \frac{C(T)}{N}, \tag{2.23}$$

and the joint law of $(X^{(1)}, \dots, X^{(k)})$ converges to the product law $\mathrm{Law}(X^{\infty,1})^{\otimes k}$ in Wasserstein-2.

Intuition. Propagation of chaos says that in a very large, exchangeable population, any finite subset of particles becomes asymptotically independent. Each one "sees" the population only through the *average* μ_t, not the identities of its neighbours. This justifies replacing μ_t^N by μ_t in theoretical analysis — and justifies simulating μ_t^N with large-but-finite N in practice. For our deep BSDE solver, we use N = batch size (typically 256); the empirical distribution is a Monte Carlo approximation of the true mean-field law.

2.6.3 Mean-field games versus mean-field control

The above is an *uncontrolled* MV-SDE: each agent's dynamics depend on the population law. In stochastic control, each agent chooses a control $\alpha^{(i)}$ to optimise their own cost. Two closely related problems emerge depending on the objective structure:

1. **Mean-field game (MFG).** Each agent is *selfish*: they optimise $J^{(i)}(\alpha^{(i)};\mu)$ treating the population law μ as *fixed*. An *equilibrium* is a fixed point: each agent's best response produces exactly the population law they assumed.
2. **Mean-field control (MFC).** A central planner optimises the *joint* cost. The planner's control shifts the whole population simultaneously; there is no fixed-point condition.

The two produce generically different solutions (Carmona–Delarue [**?**], Vol. II); MFG corresponds to Nash, MFC to Pareto. In our Chapter 5, the single-agent Nash equilibrium computed by the exact CX solver is an MFG solution; the Pareto / monopolist solution is an MFC solution. The gap between them is what tacit collusion partially recovers.

2.6.4 McKean–Vlasov BSDEs and FBSDEs

Coupling an MV-SDE with a BSDE gives a *McKean–Vlasov BSDE* (MV-BSDE) whose generator depends on the law:

$$-\mathrm{d}Y_t = f(t, X_t, Y_t, Z_t, \mu_t)\,\mathrm{d}t - Z_t\,\mathrm{d}W_t, \qquad Y_T = g(X_T, \mu_T), \quad \mu_t = \mathrm{Law}(X_t). \tag{2.24}$$

If the forward X also depends on μ_t, we have a *forward-backward* MV system (MV-FBSDE). Han, Hu and Long [**?**] extended the deep BSDE method to this setting via *fictitious play*: alternately fix μ_t and solve the resulting BSDE, then update μ_t to the empirical distribution produced by the learned solution, and iterate. Convergence (in Wasserstein distance) is proved under monotonicity assumptions.

2.7 The Cont–Xiong dealer market model

We now specialise to the setting of this dissertation. The Cont–Xiong (2024) [**?**] dealer market is a dynamic stochastic game in which each agent is a market maker quoting bid and ask half-spreads to trade with an exogenous order-flow process. It provides the cleanest available mean-field formulation of competitive market making and — crucially for our purposes — admits an *exact* fixed-point solution against which a neural solver can be validated.

2.7.1 Primitives

There are N identical dealers, each with inventory $q_t^{(i)} \in \Delta \cdot \mathbb{Z}$ (a discrete integer lattice with tick size Δ). A symmetric order-flow process generates market orders on the bid and ask sides with base arrival rates $\lambda^a, \lambda^b > 0$. The *execution probability* that a market order on side $s \in \{a, b\}$ arriving at time t is routed to dealer i is determined by the relative attractiveness of dealer i's quote versus those of the rest:

$$p_t^{(i),s} = \frac{e^{-\alpha\delta_t^{(i),s}}}{\sum_{j=1}^{N} e^{-\alpha\delta_t^{(j),s}}}, \tag{2.25}$$

where $\alpha > 0$ is the flow sensitivity. Larger α means flow is more sharply focused on the best-quoting dealer.

Intuition. Equation (2.25) is a logit. If every dealer quotes the same δ, each gets $1/N$ of the flow. If dealer i undercuts by ε, their share grows (roughly) by a factor $e^{\alpha\varepsilon}$ relative to the others. This is the source of the competitive pressure: undercutting is locally profitable but erodes the shared surplus.

Each execution on side s moves the winning dealer's inventory by Δ (up for bid fills, down for ask fills):

$$\mathrm{d}q_t^{(i)} = \Delta\big(\mathrm{d}N_t^{(i),b} - \mathrm{d}N_t^{(i),a}\big), \tag{2.26}$$

where $N_t^{(i),s}$ is a counting process with stochastic intensity $\lambda^s \cdot p_t^{(i),s}$. Each dealer collects spread revenue on each fill:

$$\mathrm{dPnL}_t^{(i)} = \delta_t^{(i),a}\,\mathrm{d}N_t^{(i),a} + \delta_t^{(i),b}\,\mathrm{d}N_t^{(i),b}. \tag{2.27}$$

2.7.2 Objective

Dealer i maximises the discounted expected profit minus a running inventory penalty:

$$J^{(i)}(\delta^{(i)};\delta^{(-i)}) = \mathbb{E}\left[\int_0^\infty e^{-rt}\big(\delta_t^{(i),a}\lambda^a p_t^{(i),a} + \delta_t^{(i),b}\lambda^b p_t^{(i),b} - \psi(q_t^{(i)})\big)\,\mathrm{d}t\right], \tag{2.28}$$

where $\psi(q) = \phi q^2$ is the quadratic penalty and $r > 0$ is the discount rate. We have taken expectations over the execution point processes, replacing $\mathrm{d}N^{(i),s}$ by its intensity $\lambda^s p^{(i),s}\,\mathrm{d}t$; this is standard because $\delta^{(i),s}$ is the revenue per fill, which commutes with the conditional expectation.

Remark 2.10 (Stationary vs finite horizon). Cont–Xiong work in the infinite-horizon discounted setting, which makes the value function $V(q)$ time-independent. The finite-horizon version has $V(t,q)$ and we use it when coupling to a horizon-dependent extension (e.g. adverse selection with a terminal liquidation penalty). The two are linked by a standard time-change argument.

2.7.3 Mean-field limit of the CX model

As $N \to \infty$, the denominator in (2.25) concentrates. Let μ_t denote the joint distribution of $(q_t^{(j)}, \delta_t^{(j),a}, \delta_t^{(j),b})$ over the population. By the law of large numbers,

$$\frac{1}{N}\sum_{j=1}^{N} e^{-\alpha\delta_t^{(j),a}} \longrightarrow \mathbb{E}_\mu\big[e^{-\alpha\Delta^a}\big] =: \bar{f}^a(\mu_t), \tag{2.29}$$

with analogous notation $\bar{f}^b$. Thus for a representative dealer with quotes (δ^a, δ^b), the execution probability simplifies:

$$p_t^a = \frac{1}{N}\cdot\frac{e^{-\alpha\delta^a}}{\bar{f}^a(\mu_t)}\cdot(1+o(1)), \tag{2.30}$$

where the $1/N$ factor reflects that each dealer's share of flow shrinks as the population grows. Absorbing the $1/N$ into the rate definition (scaling $\lambda^s \to \tilde{\lambda}^s = \lambda^s/N$), the dealer's execution intensity becomes

$$\nu^a(t) = \lambda^a e^{-\alpha\delta^a}\cdot h^a(\mu_t), \qquad h^a(\mu_t) := \bar{f}^a(\mu_t)^{-1}. \tag{2.31}$$

This is the form we work with throughout. $h^a(\mu_t) \in (0,1]$ is the *competitive factor*: in a tight-quoting population, $\bar{f}^a$ is large, so h^a is small — your fill rate is depressed. In a loose population, h^a approaches 1 and you recover near-monopolist economics.

Intuition. In the mean-field limit, the effect of the population on you reduces to a *single scalar per side* — the competitive factor $h^a(\mu_t)$. This scalar captures how aggressively the rest of the market is quoting. In the deep BSDE solver we will parameterise h^a as a small neural network of a *law embedding* $\Phi(\mu_t)$; Chapter 3 discusses which embeddings actually allow the gradient to detect distributional structure.

2.7.4 HJB equation and quote derivation

For a representative dealer in the mean-field limit, the value function $V(q)$ (infinite horizon, discount rate r) satisfies the HJB equation

$$\begin{aligned} rV(q) = \sup_{\delta^a,\delta^b \geq 0} \Big\{ & \lambda^a e^{-\alpha\delta^a} h^a(\mu)\big[\delta^a + V(q-\Delta) - V(q)\big] \\ & + \lambda^b e^{-\alpha\delta^b} h^b(\mu)\big[\delta^b + V(q+\Delta) - V(q)\big] - \psi(q)\Big\}. \end{aligned} \tag{2.32}$$

Derivation. Over a short interval $[t, t+\mathrm{d}t]$, the dealer collects expected discounted revenue from fills plus a penalty. With fill intensities $\nu^a = \lambda^a e^{-\alpha\delta^a} h^a$ and $\nu^b = \lambda^b e^{-\alpha\delta^b} h^b$, each side fires independently and at most once in $\mathrm{d}t$ (probability of two events in $\mathrm{d}t$ is $o(\mathrm{d}t)$). The value satisfies the Bellman principle:

$$\begin{aligned} V(q) = \sup_{\delta^a,\delta^b} \big\{ & \nu^a \mathrm{d}t \cdot [\delta^a + V(q-\Delta)e^{-r\mathrm{d}t}] \\ & + \nu^b \mathrm{d}t \cdot [\delta^b + V(q+\Delta)e^{-r\mathrm{d}t}] + (1 - \nu^a \mathrm{d}t - \nu^b \mathrm{d}t)V(q)e^{-r\mathrm{d}t} - \psi(q)\mathrm{d}t\big\} + o(\mathrm{d}t). \end{aligned} \tag{2.33}$$

Taylor-expanding $e^{-r\mathrm{d}t} \approx 1 - r\mathrm{d}t$, dividing by $\mathrm{d}t$, and dropping $o(1)$ terms yields (2.32). □

To solve (2.32), fix q and optimise over (δ^a, δ^b). The ask FOC is

$$\frac{\partial}{\partial \delta^a}\Big\{\lambda^a e^{-\alpha\delta^a} h^a[\delta^a - (V(q) - V(q-\Delta))]\Big\} = 0. \tag{2.34}$$

Differentiating, using the product rule,

$$\lambda^a e^{-\alpha\delta^a} h^a\big[1 - \alpha(\delta^a - (V(q) - V(q-\Delta)))\big] = 0. \tag{2.35}$$

Since the prefactor is positive, we set the bracket to zero:

$$\boxed{\delta^{a,*}(q) = \frac{1}{\alpha} + \big(V(q) - V(q-\Delta)\big).} \tag{2.36}$$

The bid side is symmetric:

$$\boxed{\delta^{b,*}(q) = \frac{1}{\alpha} + \big(V(q) - V(q+\Delta)\big).} \tag{2.37}$$

Intuition. The optimal ask is the monopolist spread $1/\alpha$ *plus* the marginal cost of losing one unit of inventory (the drop in V from q to $q-\Delta$). When you are long inventory (q large), V is decreasing, so $V(q) - V(q-\Delta) > 0$ and you quote *wider*: you want to discourage further fills on the ask side and unload the position. When short, the opposite. Equations (2.36)–(2.37) are the central formulae of this dissertation: everything we compute — whether by exact solver, neural Bellman, or deep BSDEJ — is ultimately evaluating these two finite differences of V.

Note the interesting structural property: the *total spread* $\delta^{a,*} + \delta^{b,*} = 2/\alpha + [2V(q) - V(q - \Delta) - V(q + \Delta)]$ depends only on the *curvature* of V at q, while the *skew* $\delta^{a,*} - \delta^{b,*} = V(q - \Delta) + V(q + \Delta) - 2V(q) \cdot (\text{zero}) + [V(q + \Delta) - V(q - \Delta)]$ equals the *slope* of V. For the quadratic-penalty model, V is concave and roughly parabolic in q, so the spread is near-constant in q and the skew is linear in q. We reproduce this shape numerically in Chapter 4.

2.7.5 The CX model as a BSDEJ

We now establish the theoretical connection that justifies using BSDEJ methodology on the CX model. This is the central content of the theory note [**?**], expanded here with full derivations.

Define the value process along the inventory trajectory:

$$Y_t := V(t, q_t). \tag{2.38}$$

Applying Itô's formula for jump processes (2.3) (with no diffusion term — q_t is a pure jump process),

$$\begin{aligned} \mathrm{d}Y_t = \partial_t V(t, q_{t-})\,\mathrm{d}t &+ \big[V(t, q_{t-} + \Delta) - V(t, q_{t-})\big]\mathrm{d}N_t^b \\ &+ \big[V(t, q_{t-} - \Delta) - V(t, q_{t-})\big]\mathrm{d}N_t^a. \end{aligned} \tag{2.39}$$

(We use $q_{t-} + \Delta$ for the bid jump because inventory goes up when we buy; ask jumps decrease inventory.) Introducing the jump adjoints

$$U_t^a := V(t, q_{t-} - \Delta) - V(t, q_{t-}), \qquad U_t^b := V(t, q_{t-} + \Delta) - V(t, q_{t-}), \tag{2.40}$$

compensating (2.39) by adding and subtracting the intensity-weighted jump contributions, and using the HJB equation (2.32) to re-express $\partial_t V = -\mathcal{L}^* V$, we obtain

$$-\mathrm{d}Y_t = \underbrace{\big[rY_t + \psi(q_t) - \lambda^a e^{-\alpha\delta^{a,*}} h^a[\delta^{a,*} + U_t^a] - \lambda^b e^{-\alpha\delta^{b,*}} h^b[\delta^{b,*} + U_t^b]\big]}_{=:f(t,q_t,Y_t,U_t^a,U_t^b)}\mathrm{d}t - U_t^a \mathrm{d}\tilde{N}_t^a - U_t^b \mathrm{d}\tilde{N}_t^b, \tag{2.41}$$

where $\widetilde{\mathrm{d}N_t^s} = \mathrm{d}N_t^s - \nu_t^s\,\mathrm{d}t$ are the compensated jump martingales. With the optimal quotes (2.36)–(2.37) substituted, the generator simplifies: in the optimal-ask term,

$$\delta^{a,*} + U_t^a = \big(\tfrac{1}{\alpha} + V(q) - V(q - \Delta)\big) + \big(V(q - \Delta) - V(q)\big) = \tfrac{1}{\alpha},$$

the monopolist's own markup. Analogously for the bid. So the fill-contribution terms collapse to $\nu^a \cdot 1/\alpha + \nu^b \cdot 1/\alpha$, and the final BSDEJ generator takes the Markovian form of (q_t, Y_t, U_t^a, U_t^b).

Proposition 2.11 (CX model $\leftrightarrow$ BSDEJ, stationary case)**.** *Fix $(\lambda^a, \lambda^b, \alpha, r, \phi, \Delta)$ and a population law μ. The value function $V(q)$ of the mean-field CX market maker, together with the jump adjoints $U^s(q) = V(q \pm \Delta) - V(q)$, is the unique solution of the stationary BSDEJ* (2.41) *in the sense that: the pair (V, U^a, U^b) satisfies the HJB* (2.32) *with optimal quotes* (2.36)–(2.37) *if and only if it satisfies the stationary BSDEJ.*

Proof. Forward direction: apply Itô to $V(q_t)$ as in (2.39) and compensate. The stationary HJB (2.32) gives $rV(q) =$ (generator at optimal controls), which, rearranged, is the time-independent form of (2.41). Reverse direction: starting from the BSDEJ and taking expectations term by term, the jump martingales vanish and one recovers (2.32). Uniqueness follows from Theorem 2.8. □

Remark 2.12 (Why this matters). Proposition 2.11 makes three points simultaneously:

- The CX model is not just a collection of coupled ODEs on a grid; it is a genuine BSDEJ.
- The deep BSDE methodology developed for generic BSDEJs applies directly to the CX problem.
- The jump adjoints U^a, U^b — which the neural solver will learn — have a clean economic interpretation: they are the *marginal value of an ask/bid fill*, equivalently the discrete $\partial V/\partial q$ in each direction.

2.7.6 Forward SDE approximations

For the diffusion-approximation solvers (Chapter 3), we replace the compound Poisson inventory process with its moment-matched Gaussian diffusion. Writing $\mu_q := \lambda^b f^b - \lambda^a f^a$ and $\sigma_q^2 := \lambda^a f^a + \lambda^b f^b$ (where $f^s = e^{-\alpha\delta^s}$),

$$\mathrm{d}q_t \approx \mu_q \,\mathrm{d}t + \sigma_q \,\mathrm{d}W_t^q. \tag{2.42}$$

By construction, $\mathbb{E}[\mathrm{d}q_t] = \mu_q \,\mathrm{d}t$ and $\mathrm{Var}(\mathrm{d}q_t) = \sigma_q^2 \,\mathrm{d}t$ match the first two moments of the jump increment. For high fill rates this is an accurate approximation (Donsker); for low rates it systematically understates tail risk.

Under this substitution, the BSDEJ (2.41) reduces to a classical BSDE (no U); the HJB (2.32) reduces to a second-order PDE; and the FOCs (2.36)–(2.37) are replaced by derivatives in the continuous limit:

$$\delta^{a,*} = \frac{1}{\alpha} + \frac{Z^q}{\sigma_q}, \qquad \delta^{b,*} = \frac{1}{\alpha} - \frac{Z^q}{\sigma_q}, \qquad Z^q = \sigma_q \partial_q V. \tag{2.43}$$

This is the form the original deep BSDE solver [**?**] uses, and it is the form that Chapter 3 analyses for architectural failure modes. The jump form (2.36)–(2.37) of Chapter 4 is more fundamental but harder to train directly; it requires learning U-valued networks.

2.8 Summary

We have built up the vocabulary for the rest of the dissertation:

- BSDEs and BSDEJs as backward analogues of forward SDEs, well-posed under Lipschitz conditions (Pardoux–Peng, Tang–Li).
- The nonlinear Feynman–Kac theorem identifying BSDE solutions with semilinear PIDE solutions.
- Mean-field / McKean–Vlasov formulations as $N \to \infty$ limits of exchangeable particle systems, with propagation of chaos.
- The Cont–Xiong dealer market, its HJB equation, the explicit FOC for optimal quotes, and its reformulation as a BSDEJ.

Chapter 3 turns to the question: given that BSDEJ theory justifies the approach, how do we actually solve these equations numerically using deep learning? And under what architectural conditions does that actually work?

Chapter 3

Deep BSDE Methods

Chapter 2 showed that the Cont–Xiong dealer market is a BSDEJ. That tells us *what* to solve. This chapter tells us *how*: the deep BSDE paradigm of Han, Jentzen and E [**?**], its extension to McKean–Vlasov couplings via fictitious play [**?**], the further extension to jump-driven forward dynamics, and — critically — the *architectural conditions* under which these methods actually converge to the right answer rather than producing plausible-looking artefacts.

3.1 The deep BSDE paradigm

3.1.1 Core insight

A BSDE has a Markovian solution: when the forward X is Markov and the generator f and terminal g depend on (t, X_t) only, the solution (Y_t, Z_t) has the form $Y_t = u(t, X_t)$, $Z_t = \sigma^\top \nabla u(t, X_t)$ for some function u (Theorem 2.5). If we parameterise Z_t directly by a neural network, we can train it by backpropagation through the discretised BSDE dynamics.

Han, Jentzen and E's idea is cleaner still: instead of learning Y_t and Z_t separately, *learn* Y_0 *as a scalar parameter and* Z_t *as a neural network*, then roll the BSDE *forward* in time under the Euler scheme, and minimise the mean-squared difference between the rolled-out Y_T and the known terminal $g(X_T)$.

Definition 3.1 (Deep BSDE (HJE 2018) [**?**])**.** Given a discretisation $0 = t_0 < t_1 < \cdots < t_N = T$ with $\Delta t = T/N$, define the following learnable objects:

- A scalar parameter $\theta_0 \in \mathbb{R}$ representing Y_0.
- Neural networks $Z_\theta(t_k, x) : \mathbb{R}^n \to \mathbb{R}^d$, one per time step, with parameters θ.

Simulate the forward trajectory via Euler–Maruyama:

$$X_{t_{k+1}} = X_{t_k} + \mu(t_k, X_{t_k})\Delta t + \sigma(t_k, X_{t_k})\Delta W_k, \quad \Delta W_k \sim \mathcal{N}(0, \Delta t\, I_d). \tag{3.1}$$

Roll Y forward by inverting the BSDE Euler update:

$$Y_{t_{k+1}} = Y_{t_k} - f(t_k, X_{t_k}, Y_{t_k}, Z_\theta(t_k, X_{t_k}))\Delta t + Z_\theta(t_k, X_{t_k})^\top \Delta W_k, \tag{3.2}$$

starting from $Y_{t_0} = \theta_0$. Minimise

$$\mathcal{L}(\theta_0, \theta) = \mathbb{E}\big[(Y_{t_N} - g(X_{t_N}))^2\big] \tag{3.3}$$

by Adam / SGD on Monte Carlo samples of the forward path.

Intuition. The BSDE says that (Y, Z) is the unique adapted pair making Y hit $g(X_T)$ at terminal time. Equivalently: any *other* guess for $(Y_0, Z(t, x))$ produces a rolled-out Y_T that misses $g(X_T)$. So we can train by forcing the rolled-out Y_T to match $g(X_T)$. The only constraint on Z is that it be a function of the current state X_{t_k}; we parameterise with a neural network and let gradient descent discover the right function.

A subtle but important point: the same θ_0 is the prediction for *every* sample path, because Y_0 is deterministic (it is $\mathbb{E}[g(X_T)]$ when $f \equiv 0$). The Z-networks carry all the path-dependent structure. This is why the loss (3.3) has low variance: the only randomness in $Y_T - g(X_T)$ comes from the path; the prediction Y_0 is fixed.

3.1.2 Convergence results

Beck, E, and Jentzen [**?**] established consistency: under Lipschitz f, g, as $N \to \infty$ (time discretisation refines) and network width $\to \infty$ (universal approximation holds), the learned Y_0 converges to the true $\mathbb{E}[g(X_T)]$ up to an error $O(1/\sqrt{\Delta t}) + O(\text{network error})$. Han and Long [**?**] refined this and gave empirical evidence of convergence rates. These are existence theorems; in practice the architecture — and not just the function class — determines whether gradient descent finds a good solution.

3.1.3 The BatchNorm discovery

A practical matter that is under-documented in the original HJE papers is the role of batch normalisation. In their TensorFlow 1.x reference implementation, Han, Jentzen and E wrap every hidden layer of Z_θ in a BatchNorm layer:

$$\mathrm{BN}(x) = \gamma \odot \frac{x - \mathbb{E}_{\text{batch}}[x]}{\sqrt{\mathrm{Var}_{\text{batch}}(x) + \varepsilon}} + \beta,$$

with learnable scale γ and shift β, per-feature. This is essential for convergence on the benchmark problems in [**?**]: without BN the solver often fails to find the right Y_0 even on 100-dimensional linear BSDEs where closed forms exist.

Why does BN help? At the start of training, the network weights are random; hidden activations can be wildly off-scale, and the BSDE update (3.2) amplifies any scale mismatch multiplicatively with ΔW_k. BN forces each hidden layer to have zero-mean unit-variance activations regardless of weight initialisation, stabilising the gradients. Without it, the solver is initialisation-fragile.

Intuition. BatchNorm is a *conditioning* device for the optimisation landscape. In image classification its role is well-studied; in deep BSDE it plays essentially the same role, and the gradients it cleans up are those of the BSDE roll-out rather than a feed-forward pass. It is not a modelling choice — it is a numerical-conditioning trick — but without it the method typically fails on benchmarks where it is claimed to succeed. We will see in §3.4 that this otherwise-benign trick has a catastrophic interaction with broadcast features in the mean-field setting.

3.2 Extension to McKean–Vlasov FBSDEs

For the MV-BSDE of Section 2.7, the generator depends on the law:

$$-\mathrm{d}Y_t = f(t, X_t, Y_t, Z_t, \mu_t)\mathrm{d}t - Z_t\,\mathrm{d}W_t, \qquad \mu_t = \mathrm{Law}(X_t). \tag{3.4}$$

The direct Monte Carlo approach runs into a chicken-and-egg problem: we need μ_t to compute Z_θ's output, but μ_t is the law of X_t which depends on the controls which depend on Z_θ. Han, Hu and Long [?] resolved this by *fictitious play*.

3.2.1 The fictitious-play decomposition

Definition 3.2 (Neural fictitious play for MV-FBSDEs)**.** Given an initial guess $\mu^{(0)}$ for the population law, iterate for $k = 1, 2, \ldots$:

1. (BSDE inner loop.) Fix $\mu^{(k-1)}$ in the generator. Train $(Y_0^{(k)}, Z_\theta^{(k)})$ via the HJE algorithm of Definition 3.1 treating $\mu^{(k-1)}$ as a constant input.
2. (Empirical update.) Simulate N forward trajectories under the learned controls; compute the empirical distribution $\mu^{(k)} = \frac{1}{N}\sum_{i=1}^{N} \delta_{X^{(i)}}$.
3. (Convergence check.) If $W_2(\mu^{(k)}, \mu^{(k-1)}) < \varepsilon$, stop. Otherwise repeat.

Theorem 3.3 (Han–Hu–Long [?], informal)**.** *Under a monotonicity assumption on f with respect to μ (in the sense that moving μ in the direction of the best response decreases a suitable Lyapunov functional), the fictitious-play iteration converges linearly in Wasserstein-2:*

$$W_2(\mu^{(k)}, \mu^*) \leq C\rho^k W_2(\mu^{(0)}, \mu^*), \tag{3.5}$$

for some $\rho < 1$, where μ^* is the unique fixed point.

The monotonicity assumption is stringent in general (and fails in some MFG examples) but holds for the CX model with its logit-shaped execution probabilities: an increase in the population's quote tightness decreases a representative agent's fill rate in a monotone way, inducing monotonicity of the fixed-point map.

Intuition. Fictitious play is what economists call "learning by assuming the population is as you left it last time". If you solve the BSDE thinking the population is $\mu^{(k-1)}$, and then simulate to find the actual population is $\mu^{(k)}$, you update and retry. The monotonicity assumption says this iteration contracts; in practice we see convergence within ~ 10 outer iterations to tight tolerance.

In the repo, the outer loop is in [`scripts/cont_xiong_exact.py`](../../scripts/cont_xiong_-exact.py) and [`scripts/cont_xiong_pareto.py`](../../scripts/cont_xiong_pareto.py), with Wasserstein-style difference metric logged at each iteration (see the validation plot regenerated from committed data, `plots_cx/regenerated/neural_validation.png`).

3.3 Extension to jump processes (BSDEJ solvers)

The HJE method was designed for Brownian-driven BSDEs. For our BSDEJ, we need a parameterisation of both Z_t and the jump integrand $U_t(e)$. Several strategies are viable; we describe the two used in this repository.

3.3.1 Naive per-timestep BSDEJ solver

The most direct extension of HJE is to add one more network per time step for U.

Definition 3.4 (Deep BSDEJ — per-timestep networks)**.** For a BSDEJ with a two-point jump kernel $E = \{-\Delta, +\Delta\}$ (the CX case), define:

- Scalar $\theta_0 = Y_0$.

- Networks $Z_\theta^{(k)}(x)$ and $U_\theta^{a,(k)}(x)$, $U_\theta^{b,(k)}(x)$, one triple per time step.

Simulate the forward jump-diffusion (Poisson increments $\Delta N_k^{a,b}$ with rates $\lambda^s p^s(\delta_k^{s,*})$). Roll Y forward:

$$Y_{k+1} = Y_k - f(t_k, X_k, Y_k, Z_k, U_k^a, U_k^b)\Delta t + Z_k \Delta W_k + U_k^a(\Delta N_k^a - \nu_k^a \Delta t) + U_k^b(\Delta N_k^b - \nu_k^b \Delta t), \quad (3.6)$$

where the Z, U are evaluated at the network outputs. Minimise $\mathbb{E}[(Y_N - g(X_N))^2]$.

This was the first BSDEJ variant implemented in [`solver_cx_bsdej.py`](../../solver_cx_bsdej.py). The disadvantage is that each time step gets its own independent networks, so the total parameter count scales linearly with N (number of discretisation steps). For $N = 20$ steps with modest networks, this already blows up to millions of parameters on a problem whose underlying solution is a 1-D function $V(q)$.

3.3.2 Shared-weights + warm-start BSDEJ solver

A better architecture exploits the stationary structure of the (infinite-horizon or periodic) problem: the optimal (V, U^a, U^b) does not depend on t. We share weights across time steps and initialise the networks from the output of the fictitious-play exact solver.

Definition 3.5 (BSDEJ with shared weights + warm start)**.** Replace the per-step $Z_\theta^{(k)}, U_\theta^{s,(k)}$ of Definition 3.4 with *a single network each*:

$$Z_\theta(x), \quad U_\theta^a(x), \quad U_\theta^b(x).$$

Initialise the weights so that the network outputs agree (to ε-precision) with the fictitious-play approximation of $V(q \pm \Delta) - V(q)$ at the grid points (this is the warm start). Run the HJE-style training loop with $(Y_0, \theta_Z, \theta_{U^a}, \theta_{U^b})$ as learnable parameters.

The implementation is in [`solver_cx_bsdej_shared.py`](../../solver_cx_bsdej_shared.py). On the stationary CX benchmark, this solver achieves a spread error of 2.6% against the exact Algorithm 1, versus $> 10\%$ without the warm start (see the key results table in the main README).

Intuition. The warm-start trick is essential in practice. Starting from random weights, the BSDEJ loss landscape has many plateaus and the gradient is dominated by the jump compensator noise. Starting from the exact solution's values means the gradient signal can immediately detect and correct small discrepancies rather than having to discover the basin from scratch. In this sense the neural solver is doing a *continuous refinement* of the discrete tabular solver, not a from-scratch optimisation.

The shared-weights choice is defensible for the stationary problem because the true solution is time-homogeneous. For a genuinely time-inhomogeneous problem (horizon-dependent ψ, terminal liquidation, etc.) per-step networks recover flexibility.

3.3.3 Compensated martingale ablation

A subtle implementation detail worth flagging: the roll-out (3.6) must use *compensated* increments $\Delta N_k^s - \nu_k^s \Delta t$. Using the raw jump counts ΔN_k^s instead introduces a systematic bias equal to the mean jump compensator, which compounds over N steps to produce catastrophic errors. The [`scripts/compensated_martingale_ablation.py`](../../scripts/compensated_martingale_ablation.py) script shows: with raw jumps, error is 264%; with compensation, 2.7%. This is the Wang et al. (2023) [?] fix for jump-BSDEs and reflects a more subtle point about the BSDEJ roll-out than most practitioners appreciate. The theoretical reason is that

the BSDEJ martingale part is driven by the *compensated* measure $\tilde{N}$, not N; using raw counts corresponds to a different equation with a nonzero drift in the martingale direction.

3.4 Architectural failure modes of MV deep BSDE solvers

This section presents the diagnostic findings from [?] in full, with derivations and numerical evidence. These are the "three failure modes" that silently suppress mean-field signals in deep BSDE architectures; each is sufficient on its own to produce a "no mean-field effect" non-result when in fact the effect is present but architecturally invisible.

3.4.1 Mean-field conditioning

Recall from §2.7.3 that the mean-field generator depends on the law μ_t through a competitive factor $h^a(\mu_t)$. To feed μ_t into a neural network, we need a finite-dimensional encoding $\Phi(\mu_t) \in \mathbb{R}^d$. Four encoders are implemented in the repo ([`equations/law_encoders.py`](../../equations/law_encoders.py)):

Encoder	Features	Dimension
Moments	$\mathbb{E}[q], \mathrm{Var}[q], \mathrm{Skew}[q], \mathbb{E}\lvert q\rvert, \max\lvert q\rvert, \sigma_q$	6
Quantiles	$\mathbb{E}[q]$ plus 5 quantiles $Q_{0.1,0.25,0.5,0.75,0.9}$	6
Histogram	soft Gaussian bins on $[-q_{\max}, q_{\max}]$	20
DeepSets	$\rho\big(\frac{1}{N}\sum_i \psi(x_i)\big)$, ψ and ρ learned	16

Each encoder's output $\Phi(\mu_t)$ is fed into (i) a *competitive factor network* $h_\theta(\Phi) \in (0, 1]$ that appears in the generator, and (ii) the subnet that produces Z_θ, as a per-particle input concatenated with X_t. In the broken pre-fix implementation, one of the two paths was severed or degraded. Three independent failures follow.

3.4.2 Failure 1: generator bypass

The first failure is an implementation bug, cleanly separable from architectural issues. The MV class inherited f_{tf} — the BSDE generator — from its non-MV parent. The parent's generator used a scalar *moment proxy* $\hat{h}(t, \bar{q}_t, \bar{\delta}_t)$ based on first-order statistics, not the full law embedding $\Phi(\mu_t)$. As a result, the BSDE dynamics (3.2) did not depend on $\Phi(\mu_t)$ — only on the moment proxy.

Consequence: the gradient of the loss $\mathcal{L}$ with respect to the law-encoder parameters is

$$\frac{\partial \mathcal{L}}{\partial \theta_\Phi} = \frac{\partial \mathcal{L}}{\partial Y_T} \cdot \frac{\partial Y_T}{\partial \Phi} \cdot \frac{\partial \Phi}{\partial \theta_\Phi}. \tag{3.7}$$

If the BSDE roll-out does not depend on Φ, then $\partial Y_T/\partial \Phi = 0$ and no gradient flows back. The law encoder is trained to do *nothing* and converges to a trivial output.

The fix, implemented in [`equations/contxiong_lob_mv.py`](../../equations/contxiong_lob_mv.py) lines 23–124, overrides f_{tf} in the MV class to use the law-embedding-based competitive factor network $h(\Phi)$ directly in execution probabilities. This reconnects the gradient path from the loss back to the encoder.

3.4.3 Failure 2: BatchNorm erasure of broadcast features

After fixing Failure 1, we found that the law embedding was *still* being ignored. Reason: at a given time step, $\Phi(\mu_t)$ is computed once from the empirical population and *broadcast to*

all N particles in the batch, because every particle in the population sees the same μ_t. This produces the following structure in the batch input to the subnet:

$$\text{batch input} = \Big[\underbrace{(X^{(1)}, \Phi(\mu_t))}_{\text{particle 1}}, \underbrace{(X^{(2)}, \Phi(\mu_t))}_{\text{particle 2}}, \ldots, \underbrace{(X^{(N)}, \Phi(\mu_t))}_{\text{particle } N}\Big]. \tag{3.8}$$

The X-coordinates vary across the batch (every particle has its own state). The Φ-coordinates are *identical* across the batch (the law is the same object broadcast to everyone).

BatchNorm (§3.1.3) computes per-feature statistics across the batch dimension:

$$\mathrm{BN}(x)_j = \gamma_j \frac{x_j - \bar{x}_j}{\sqrt{\mathrm{Var}_j + \varepsilon}} + \beta_j.$$

For the Φ-coordinates, x_j is constant across the batch, so $\bar{x}_j = x_j$ and $\mathrm{Var}_j = 0$. The numerator is identically zero; the output is β_j (a learnable constant). **BatchNorm has annihilated the law embedding.** Worse, this erasure propagates through every downstream BatchNorm layer: the residual $x_j - \bar{x}_j$ stays zero no matter how the layers transform x_j.

Proposition 3.6 (BN erasure of broadcast features)**.** *Let ℓ be a feedforward composition of BatchNorm + affine + nonlinearity layers applied to a batch input in which the j-th feature is constant across the batch. Then the j-th feature's contribution to the output is constant across the batch at every layer (and identically zero up to β-shifts after the first BN).*

Proof. At the input BN, the j-th coordinate has $\mathrm{Var}_j = 0$; the normalised output is β_j (a batch-constant). Linearly combining a batch-constant with the (varying) other features produces a batch-constant contribution on the j-th axis. Applying the next nonlinearity and BN, the j-th contribution remains batch-constant because BN's mean subtraction on a batch-constant gives zero. Iterate. □

Intuition. A BatchNorm layer is "noise-seeking": it keeps only cross-batch variance. Features that are broadcast (and hence have zero cross-batch variance by construction) are thrown away. The law embedding $\Phi(\mu_t)$ is exactly such a feature. This is the central diagnostic finding of the preprint: mean-field BSDE solvers using standard BN-heavy architectures (as recommended by HJE for their conditioning benefits) are *structurally incompatible* with broadcast law features.

The fix, in [`solver.py`](../../solver.py) lines 31–82 (`MeanFieldSubNet`), is a *two-stream architecture*: split the input into a state stream (with BN for conditioning) and a law stream (without BN), process separately, and combine only at the output. Explicitly, let BN denote BatchNorm and MLP_ϕ a batch-normalised MLP:

$$h_{\text{state}} = \mathrm{MLP}_\phi(\mathrm{BN}(X)), \tag{3.9}$$

$$h_{\text{law}} = \mathrm{MLP}(\Phi) \quad \text{(no BN)}, \tag{3.10}$$

$$Z_\theta(X, \Phi) = h_{\text{state}} + h_{\text{law}}. \tag{3.11}$$

The state stream retains the conditioning benefit of BN; the law stream passes through unaffected. The additive combination is one choice; FiLM conditioning (below) is an alternative that allows richer state–law interactions.

3.4.4 Failure 3: representation collapse under DeepSets + symmetric inputs

Even with Failure 1 and 2 fixed, one encoder — DeepSets — systematically under-performs. We analyse the cause.

The DeepSets encoder is

$$\Phi(\mu) = \rho\left(\frac{1}{N}\sum_{i=1}^{N}\psi(x_i)\right), \tag{3.12}$$

where $\psi : \mathbb{R} \to \mathbb{R}^d$ and $\rho : \mathbb{R}^d \to \mathbb{R}^{d_{\text{embed}}}$ are MLPs. Zaheer et al. [**?**] proved that every permutation-invariant function of a multiset of bounded size is of the form (3.12). This is a universal approximation result: DeepSets *can* represent any law-invariant summary, including variance.

However, universal approximation is a statement about the function class, not about whether gradient descent finds the right member starting from a random initialisation. At initialisation, ψ is a random MLP. For a symmetric input distribution (say, zero-mean q), the per-particle outputs $\psi(q_i)$ are approximately symmetric around the origin. The *mean pooling* $\frac{1}{N}\sum_i \psi(q_i)$ then produces positive and negative contributions that *cancel*. The resulting pooled vector is near-zero, and ρ's input has near-zero cross-population variance — so the DeepSets encoder cannot distinguish populations with different spreads.

Proposition 3.7 (DeepSets symmetric-cancellation)**.** *Let ψ be a random ReLU MLP with symmetric weight initialisation (e.g. Kaiming uniform). Let μ_1, μ_2 be two zero-mean symmetric distributions on $\mathbb{R}$ with different variances. Then $\mathbb{E}_\psi[\|\Phi(\mu_1) - \Phi(\mu_2)\|^2] = O(1/N)$ in the Monte Carlo sample size N.*

Proof idea. Write $\psi(q) = \psi_{\text{even}}(q) + \psi_{\text{odd}}(q)$ where $\psi_{\text{even}}, \psi_{\text{odd}}$ are the symmetric/antisymmetric parts under $q \mapsto -q$. For random symmetric initialisation, the induced decomposition has zero mean in the odd part averaged over symmetric inputs. The mean pooling $\frac{1}{N}\sum_i \psi(q_i)$ restricted to the odd part concentrates around zero by the law of large numbers, regardless of the variance of μ. Hence $\Phi(\mu_1)$ and $\Phi(\mu_2)$ differ only in the even part's response, which is identical if μ_1, μ_2 share their even moments. For zero-mean symmetric distributions this means the first deviation is the variance, which the pooling collapses to the expectation of an even ψ_{even} — a single scalar that tends to be the same for any reasonable μ. □

Empirically (preprint [**?**], Table 2), for μ_1 with $\sigma = 0.1$ and μ_2 with $\sigma = 5.0$:

Encoder	$\|\Phi(\mu_1) - \Phi(\mu_2)\|$	$\cos(\Phi(\mu_1), \Phi(\mu_2))$
DeepSets	0.003	**1.000**
Quantiles	9.84	varies
Moments	**24.98**	varies

DeepSets sees two populations with 50× variance difference as *essentially identical*. Moments and quantiles, which explicitly compute variance-sensitive features, do not suffer this failure.

Intuition. Universal approximation guarantees the existence of a DeepSets embedding that correctly discriminates by variance — one would need $\psi(q) \approx (q, q^2)$ and ρ to compute $\mu_2 - \mu^2$. But training does not find this ψ in practice: the gradient signal is symmetric around the starting point, and symmetry preserves itself under gradient descent. The lesson generalises:

for mean-field problems, prefer *explicit variance features* over learned pooling. It is faster and more robust.

3.4.5 Corrected architecture: putting the fixes together

After the three fixes — (1) connect f to Φ, (2) two-stream subnet, (3) Moments or Quantiles encoder — the solver recovers measurable mean-field effects. The key empirical results, reproduced from the preprint and confirmed in `archive/FINDINGS.md`:

- The learned competitive factor $h(\Phi(\mu))$ varies by $4\times$ across population shapes: $h = 0.442$ for a narrow population ($\sigma = 0.1$), $h = 0.109$ for a wide population ($\sigma = 5.0$).
- Optimal ask quotes at $q = 0$ shift from $\delta_a = 0.696$ (narrow) to $\delta_a = 0.334$ (wide) — a $2\times$ change.
- Quote skew $\delta_a - \delta_b$ flips sign between narrow and wide populations at $q = 2$ (long inventory): slightly positive for narrow, sharply negative for wide.

These effects were *always present* in the mathematics; the earlier "no effect" results were artefacts of the architecture's inability to propagate the signal. The diagnostic contribution is clean: the failure modes are easy to identify a priori once you know what to look for, and fixable with minor architecture changes.

3.4.6 FiLM conditioning as an alternative

An alternative to the additive two-stream is *FiLM* (Feature-wise Linear Modulation, Perez et al. [**?**]), which modulates the state stream multiplicatively by the law embedding:

$$(\gamma(\Phi), \beta(\Phi)) = \mathrm{MLP}_\phi(\Phi), \tag{3.13}$$

$$h_{\text{state}} = \mathrm{MLP}_{\text{pre}}(\mathrm{BN}(X)), \tag{3.14}$$

$$h_{\text{modulated}} = \gamma(\Phi) \odot h_{\text{state}} + \beta(\Phi), \tag{3.15}$$

$$Z_\theta(X, \Phi) = \mathrm{MLP}_{\text{post}}(h_{\text{modulated}}). \tag{3.16}$$

The difference between FiLM and additive is whether the law can *modulate* or merely *shift* the state response. FiLM allows non-trivial interaction $\partial^2 Z/(\partial X \cdot \partial \Phi) \neq 0$, which the additive form cannot express. For the CX problem, empirically both work and perform comparably; FiLM is marginally better on high-ϕ regimes where the quote-inventory interaction is steep.

3.5 Summary

We now have a complete numerical methodology:

- Deep BSDE (HJE): learn Y_0 scalar + Z_t network, minimise terminal mismatch.
- Extensions to MV couplings (fictitious play, Theorem 3.3) and jumps (BSDEJ with U-networks).
- Three architectural failure modes: generator bypass, BN erasure, DeepSets collapse. All fixable.

The next two chapters apply this machinery. Chapter 4 uses it for the neural Bellman validation and Q-scaling experiments. Chapter 5 extends to the decentralised multi-agent setting and uncovers tacit collusion.

Chapter 4

The Neural Bellman Solver

Chapters 2–3 developed the theory. This chapter delivers the central computational result of the dissertation: a neural solver that learns the Cont–Xiong value function and optimal quotes, validated to sub-one-percent accuracy against the exact tabular Algorithm 1 of Cont and Xiong, and extended to regimes (larger inventory grids, continuous inventory) where the tabular method becomes intractable.

The chapter proceeds in order of increasing generality. We first describe the *stationary neural Bellman solver* — a value-iteration-style method that exploits the time-homogeneity of the infinite-horizon CX problem. We then validate it against the exact solver. We then scale the inventory domain Q to sizes where the exact method stalls. Finally we extend to continuous inventory using a deep BSDE with $Z \neq 0$.

4.1 Stationary formulation

The infinite-horizon discounted CX problem has a time-independent value function $V(q)$ satisfying the stationary HJB equation (2.32):

$$rV(q) = \sup_{\delta^a,\delta^b} \big\{\nu^a(q,\delta^a)[\delta^a - (V(q) - V(q-\Delta))] + \nu^b(q,\delta^b)[\delta^b - (V(q) - V(q+\Delta))] - \psi(q)\big\}.$$

Substituting the optimal quotes from (2.36)–(2.37) gives a fixed-point equation on the function $V : \Delta\mathbb{Z} \to \mathbb{R}$. The exact Algorithm 1 of Cont–Xiong solves this by a contraction mapping on the inventory grid. The neural alternative parameterises V as a neural network and minimises a *Bellman residual* loss.

Definition 4.1 (Neural Bellman solver, stationary)**.** Let $V_\theta : [-Q_{\max}, Q_{\max}] \to \mathbb{R}$ be a small MLP. For a batch of inventories $\{q_1, \ldots, q_B\}$ sampled uniformly from the inventory range, compute the Bellman target $T(V_\theta)(q)$:

$$T(V_\theta)(q) = \frac{1}{r}\cdot\Big[\nu^{a,*}(q)[\delta^{a,*}(q)-(V_\theta(q)-V_\theta(q-\Delta))]+\nu^{b,*}(q)[\delta^{b,*}(q)-(V_\theta(q)-V_\theta(q+\Delta))]-\psi(q)\Big], \tag{4.1}$$

where $\delta^{a,*}, \delta^{b,*}$ are evaluated from (2.36)–(2.37) using the current V_θ, and $\nu^{s,*}$ is the intensity at the optimal quote. Minimise

$$\mathcal{L}(\theta) = \frac{1}{B}\sum_{k=1}^{B}\big(V_\theta(q_k) - T(V_\theta)(q_k)\big)^2. \tag{4.2}$$

The method is essentially *value iteration*: at each gradient step, V_θ is nudged towards the target $T(V_\theta)$, which itself is computed from V_θ (self-bootstrap). Under standard Banach-fixed-point arguments, if the target map T is a contraction in the sup norm (which it is for $r > 0$), the iteration converges to the unique fixed point.

Intuition. The stationary neural Bellman solver bypasses the forward-simulation machinery of the HJE method entirely. Instead of rolling out a BSDE in time and minimising terminal mismatch, we directly enforce the HJB fixed-point equation at uniformly-sampled inventory points. This is simpler, faster, and more accurate in the stationary regime, because:

- No time discretisation error (no Euler scheme, no BSDE roll-out).
- No Monte Carlo variance from Brownian increments.
- The loss is the *direct* Bellman residual, not a proxy.

The trade-off: the method assumes time-homogeneity. For horizon-dependent problems (terminal liquidation, time-varying coefficients) we fall back on the deep BSDE / BSDEJ methods of Chapter 3.

The implementation is in [`solver_cx.py`](../../solver_cx.py) with a two-layer MLP and batch size $B = 512$. Training takes ~ 1000 gradient steps — a few seconds on a modern GPU.

4.1.1 Neural fictitious play: coupling to the population

For a mean-field problem, the intensities $\nu^{a,*}, \nu^{b,*}$ depend on the competitive factor $h^s(\mu)$ of the population. We use the fictitious-play iteration of Definition 3.2: in the inner loop, fix μ and solve the Bellman equation for a representative agent; in the outer loop, simulate trajectories under the learned quotes and update μ to the empirical distribution.

The outer loop typically converges within 10 iterations, measured by a Wasserstein-style difference metric. See the regenerated validation plot `plots_cx/regenerated/neural_validation.png` (right panel) for the convergence curve on the $N = 2$ symmetric Nash equilibrium.

4.2 Validation against the exact solver

The Cont–Xiong model in the stationary regime admits an *exact* tabular solution via fictitious play on the inventory grid. Algorithm 1 of [**?**] iterates: given current population quotes $\bar{\delta}^{a,(k)}, \bar{\delta}^{b,(k)}$, each agent solves their single-agent Bellman equation by backward-induction-style linear system solve; the next iterate $\bar{\delta}^{a,(k+1)}, \bar{\delta}^{b,(k+1)}$ is the average of the resulting best responses. The implementation is in [`scripts/cont_xiong_exact.py`](../../scripts/cont_xiong_exact.py) and gives the ground truth we validate against.

4.2.1 Benchmark setup

We use the repo's default parameters:

The exact solver converges in 7 outer iterations to $V(0) = 33.408$ and spread $\delta^a(0) + \delta^b(0) = 1.478$; it yields the full inventory profiles of $\delta^{a,*}(q), \delta^{b,*}(q), V(q)$ reported in the committed JSON files at `archive/old_results/results_cx_exact/nash_N2.json`.

Parameter	Value	Meaning
$\lambda^a = \lambda^b$	2	base arrival rate per side
α	implicit via FOC	flow sensitivity
r	0.01	discount rate
ϕ	0.005	inventory penalty coefficient
Δ	1	inventory tick size
Q	5	inventory range $\{-Q, \ldots, +Q\}$

4.2.2 Results: agreement with exact solver

The neural Bellman solver, trained with 10 outer fictitious-play iterations and 1000 inner gradient steps per outer iteration, produces: And at every grid point in $\{-5, -4, \ldots, 5\}$ the ask

Quantity at $q = 0$	Exact (Algorithm 1)	Neural Bellman	Relative error
Spread $\delta^a + \delta^b$	1.4779	1.4662	0.79%
Value $V(q = 0)$	33.408	33.835	1.28%

and bid profiles agree to within $\sim 1\%$; see the leftmost panel of `plots_cx/regenerated/neural_validation.png`. The outer fictitious-play loop converges geometrically at rate ~ 0.45 per iteration (rightmost panel, log-scale).

4.2.3 Q-scaling: where the tabular method stalls

The exact Algorithm 1 solves a linear system of size $\sim Q^3$ (cubic in the inventory range) at each outer iteration. For moderate Q this is cheap, but by $Q = 50$ it becomes numerically ill-conditioned: the boundary values dominate the interior, and floating-point round-off contaminates the fictitious-play update. In contrast, the neural Bellman solver scales *linearly* in batch size, and interpolates between grid points, so increasing Q is essentially free.

The repo ships a direct-V variant ([`scripts/q_scaling_direct_v.py`](../../scripts/q_scaling_direct_v.py)) that learns V on a finer inventory grid by L-BFGS on the Bellman residual, with a boundary-handling patch that stabilises the update at $\pm Q$. With the boundary fix, the direct-V method achieves a spread error of 0.0001% at $Q = 5$ (essentially machine precision) and can be pushed to $Q = 50$ without qualitative degradation. This is a *consistency* check: the neural method converges to the exact solution if given enough training capacity; any residual error in the standard solver reflects optimisation budget, not modelling bias.

4.2.4 N-scaling: mean-field regime confirmed

For increasing N (number of agents), propagation of chaos (Theorem 2.9) predicts that the symmetric Nash value function approaches a deterministic limit at rate $O(1/\sqrt{N})$. The [`scripts/bsdej_n_scaling.py`](../../scripts/bsdej_n_scaling.py) experiment runs fictitious play at $N \in \{2, 5, 10, 20, 50, 5000\}$ and confirms this rate. The mean-field limit is reached to within numerical precision by $N \gtrsim 100$. This is both a consistency check on the mean-field formulation and a justification for using mean-field-based theoretical predictions in analyses of moderately-large N games.

4.3 Extension to continuous inventory

The discrete-inventory CX problem assumes fills come in unit-sized lots. Real market microstructure has continuous fill sizes (roughly, a geometric distribution of lot sizes). A cleaner model uses the *diffusion approximation* (2.42): the jump inventory process is replaced by its moment-matched Gaussian diffusion. Under this substitution the value function $V(q)$ is defined on all of $\mathbb{R}$ rather than the discrete lattice $\Delta\mathbb{Z}$, and the finite-difference jump adjoints of Chapter 2 become continuous derivatives.

4.3.1 From FOC to continuous quotes

Recall the diffusion-approximation FOC from (2.43):

$$\delta^{a,*}(q) = \frac{1}{\alpha} + \frac{Z^q(q)}{\sigma_q(q)}, \qquad \delta^{b,*}(q) = \frac{1}{\alpha} - \frac{Z^q(q)}{\sigma_q(q)}, \tag{4.3}$$

with $Z^q(q) = \sigma_q(q)\partial_q V(q)$. In the continuous-inventory BSDE, the adjoint $Z_t = \sigma_q \partial_q V$ is a function of q alone (time-homogeneous in the stationary regime) and is learned directly by a neural network. The implementation is in [`solver_cx_bsde_diffusion.py`](../../solver_cx_bsde_diffusion.py).

4.3.2 BSDE formulation

With continuous q, the value function satisfies the stationary second-order HJB:

$$rV = \sup_{\delta^a,\delta^b}\Big\{\mu_q\partial_q V + \tfrac{1}{2}\sigma_q^2\partial_q^2 V + \lambda^a e^{-\alpha\delta^a}h^a\delta^a + \lambda^b e^{-\alpha\delta^b}h^b\delta^b - \psi(q)\Big\}, \tag{4.4}$$

which — via Feynman–Kac (Theorem 2.5) — is equivalent to a classical (non-jump) BSDE for (Y, Z). We train Z directly as a neural network and recover V by integrating the stationary identity $\partial_q V = Z/\sigma_q$ (or equivalently by a two-network setup V_θ, Z_θ with a consistency constraint).

4.3.3 Validation

On the same parameters as §4.2 but treating inventory as continuous, the BSDE diffusion solver recovers the discrete Algorithm 1 answer to within $\sim 2\%$ error on value and $\sim 1\%$ on spread. The slightly higher error relative to the tabular-discrete neural Bellman reflects the additional modelling approximation: diffusion approximation of the compound Poisson inventory is accurate at high arrival rates, less so at moderate rates ($\lambda = 2$ is borderline).

The continuous-inventory solver is the natural workhorse for extensions that genuinely exercise continuity (common noise on the price process, multi-asset formulations) and is used in Chapter 5 to check MADDPG outcomes against the mean-field benchmark in a continuous-action regime.

4.4 Finite-horizon BSDEJ solver

For completeness we describe the finite-horizon solver of [`solver_cx_bsdej_shared.py`](../../solver_cx_bsdej_shared.py), which is the deep BSDEJ of Definition 3.5 applied to the time-inhomogeneous CX problem with terminal liquidation penalty $g(q_T) = -\psi(q_T)$. This is the solver that validates the theoretical identification of the CX model with a BSDEJ (Proposition 2.11): it is the direct numerical realisation of that equation.

4.4.1 Architecture

Three shared-weight networks:

- $Z_\theta(q) : \mathbb{R} \to \mathbb{R}$: the diffusion adjoint (zero in the pure-jump formulation, non-zero if price volatility is included).
- $U^a_\theta(q), U^b_\theta(q) : \mathbb{R} \to \mathbb{R}$: the two jump adjoints.

Plus the scalar $Y_0 = \theta_0$. Forward simulation uses compound Poisson increments for q and a price process for S (optional). Backward roll-out (3.6) minimises the terminal mismatch.

4.4.2 Warm-start from fictitious play

A from-scratch initialisation often produces a large basin around suboptimal solutions. The warm start uses the converged fictitious-play approximations $\hat{V}^{(\infty)}(q)$ and $\hat{U}^{s,(\infty)}(q) = \hat{V}^{(\infty)}(q \pm \Delta) - \hat{V}^{(\infty)}(q)$ as network-pretraining targets: a few hundred gradient steps minimising

$$\|U^a_\theta - \hat{U}^{a,(\infty)}\|^2_{L^2(\text{grid})} + \|U^b_\theta - \hat{U}^{b,(\infty)}\|^2_{L^2(\text{grid})}$$

places the networks in the basin of attraction of the true solution. After warm start, the BSDEJ loss is minimised over $(Y_0, \theta_Z, \theta_{U^a}, \theta_{U^b})$.

4.4.3 Validation

On the $Q = 5, T = 20$ benchmark, the BSDEJ shared-weights solver achieves:

- Spread error at $q = 0$: 2.6% (vs exact).
- Without warm start: $> 10\%$ error, frequent divergence.
- Without compensated increments ($\mathrm{d}N$ instead of $\mathrm{d}N - \nu \mathrm{d}t$): 264% error, catastrophic bias (compensated-martingale ablation, [`scripts/compensated_martingale_ablation.py`](../../scripts martingale_ablation.py)).

The 2.6% headline error is larger than the 0.79% of the stationary neural Bellman solver, reflecting (a) the finite-horizon problem is a different (harder) instance and (b) the BSDEJ method pays the time-discretisation-plus-Monte-Carlo price that the stationary Bellman solver avoids. This is the right trade-off: the BSDEJ solver handles time-inhomogeneity that the Bellman solver cannot.

4.5 Summary of validated numerical results

Collecting the numerical evidence:

Method	Quantity	Error vs exact
Neural Bellman (stationary, $Q = 5$)	spread at $q = 0$	0.79%
Neural Bellman (stationary, $Q = 5$)	$V(q = 0)$	1.28%
Direct-V (boundary-patched, $Q = 5$)	spread at $q = 0$	0.0001%
BSDEJ shared + warmstart ($T = 20$)	spread at $q = 0$	2.6%
BSDEJ without warmstart ($T = 20$)	spread at $q = 0$	$> 10\%$ (fails)
BSDEJ without compensated martingale	spread at $q = 0$	264% (catastrophic)
Mean-field $N \to \infty$ rate	Wasserstein(V_N, V_∞)	$O(1/\sqrt{N})$, confirmed to $N = 5000$

The headline claim — that deep BSDE methods can achieve sub-percent accuracy on validated benchmarks in the CX market-making regime — is borne out across four independent methods, each with its own trade-offs. The methods are now ready to be applied to regimes where the exact Algorithm 1 breaks down: the multi-agent extension in Chapter 5, the high-Q extension, the continuous-inventory extension, and the horizon-dependent extension.

Chapter 5

Nash Equilibrium and Tacit Collusion

Chapter 4 established a validated solver for the mean-field version of the CX problem — one representative agent best-responding to a fixed population law. This chapter turns to the *genuine multi-agent* setting, in which a finite number N of strategic dealers simultaneously learn policies and influence each other. Three equilibrium concepts become relevant, and empirically they are all distinct: the single-shot *Nash equilibrium*, the joint-optimal *Pareto* (cartel) outcome, and a *tacit-collusion* regime that emerges under decentralised gradient-based learning.

Our empirical contribution is a statistical confirmation that MADDPG-trained dealers converge to spreads strictly above Nash: at $N = 2$, 15 of 20 independent Hamilton-cluster seeds do so ($p = 0.002$, one-sided binomial test); at $N = 5$, 5 of 5 seeds do. This chapter derives the Nash and Pareto benchmarks from first principles and discusses the mechanisms behind the observed collusion.

5.1 Equilibrium concepts

Fix the market parameters (arrival rates, flow sensitivity, discount, penalty). An N-agent strategy profile is $(\pi^{(1)}, \ldots, \pi^{(N)})$ where each $\pi^{(i)}$ maps the agent's state (at minimum their inventory $q^{(i)}$) to a pair of half-spreads $(\delta^{(i),a}, \delta^{(i),b})$. Each agent's value is the expected discounted profit:

$$J^{(i)}(\pi^{(i)}; \pi^{(-i)}) = \mathbb{E}\left[\int_0^\infty e^{-rt}\big(r_t^{(i),a} + r_t^{(i),b} - \psi(q_t^{(i)})\big)\,\mathrm{d}t\right], \tag{5.1}$$

where $r_t^{(i),s}$ is agent i's expected revenue rate on side s.

5.1.1 Nash equilibrium

Definition 5.1 (Nash equilibrium)**.** A profile $(\pi^{*,1}, \ldots, \pi^{*,N})$ is a Nash equilibrium if for every i and every $\pi^{(i)}$,

$$J^{(i)}(\pi^{*,i}; \pi^{*,-i}) \geq J^{(i)}(\pi^{(i)}; \pi^{*,-i}). \tag{5.2}$$

Equivalently: at the equilibrium profile, no single agent can profitably deviate unilaterally. For the symmetric CX market — where all agents share the same objective functional and the dynamics are exchangeable — there is a unique *symmetric* Nash equilibrium in which every agent follows the same policy π^*. Cont and Xiong [?] proved this and gave Algorithm 1 for computing π^* by fictitious play on the inventory grid.

5.1.2 Pareto optimum (cartel)

Definition 5.2 (Pareto optimum)**.** A profile is Pareto-optimal if no alternative profile makes every agent weakly better off and some agent strictly better off.

For the symmetric CX market, the efficient Pareto profile corresponds to the *cartel* outcome: all agents agree to quote the monopolist spread $\delta^a = \delta^b = 1/\alpha$ and share flow equally. This maximises joint revenue because the logit share equalises at $1/N$, while the expected revenue per fill (the monopolist's $\delta \cdot e^{-\alpha\delta} = 1/(\alpha e)$) is the maximum achievable. The Pareto spread is therefore wider than the Nash spread.

5.1.3 Nash vs Pareto: the prisoner's dilemma structure

The CX market is a prisoner's-dilemma in the following sense. Consider any agent facing the Pareto profile. Their best response is *not* to continue cooperating at $\delta = 1/\alpha$; it is to undercut slightly and capture a larger flow share at a small reduction in per-fill margin. Iterated undercuts by all agents drive spreads below the Pareto value down to the Nash point, where no further undercut is profitable. At Nash, each agent is *locally* optimal but the population is *jointly* suboptimal (everyone would be better off agreeing to cooperate).

5.1.4 Exact benchmarks from the committed JSONs

Running the exact solvers with the repo's default parameters produces the following:

Regime	$\delta^a(0)$	$\delta^b(0)$	Spread at $q = 0$	$V(0)$
Pareto / monopolist	0.797	0.797	1.593	28.02
Nash ($N = 2$)	0.739	0.739	**1.478**	33.41
Nash ($N = 5$)	0.739	0.739	1.478	33.41

Two observations:

1. The Pareto spread exceeds the Nash spread by $\sim 7\%$: this is the coordination surplus that cooperation would extract.
2. Nash at $N = 5$ equals Nash at $N = 2$: at $q = 0$ the symmetric equilibrium is structurally invariant to N because of the normalisation λ^s/N in the MF limit. This may look surprising but is expected (Cont–Xiong [**?**], Prop. 3.2).

The regenerated plot `plots_cx/regenerated/nash_pareto_spreads.png` visualises the quote profiles and spread comparison across the inventory range.

5.2 Multi-agent deep reinforcement learning

Solving the single-shot Nash gives the *equilibrium outcome* if agents could commit to equilibrium strategies. But real agents — whether human traders or automated market-making algorithms — do not compute fixed points; they *learn*. What learning dynamics actually converge to depends on the algorithm, the initialisation, and the gradient flow. This section describes the Multi-Agent Deep Deterministic Policy Gradient (MADDPG) algorithm of Lowe et al. [**?**] applied to the CX market, as implemented in [`solver_cx_multiagent.py`](../../solver_cx_multiagent.py).

5.2.1 DDPG for single-agent continuous control

Deep Deterministic Policy Gradient (DDPG, Silver et al. [**?**]) is the actor-critic RL method for continuous action spaces. For a single agent:

- **Actor** $\pi_\theta(s) : \mathcal{S} \to \mathcal{A}$: deterministic policy mapping states to actions.
- **Critic** $Q_\phi(s, a) : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$: state-action value function.

Training alternates two steps:

1. **Critic update** (TD learning): given a transition (s, a, r, s') from a replay buffer,

$$y = r + \gamma Q_{\phi^-}(s', \pi_{\theta^-}(s')), \qquad \phi \leftarrow \phi - \eta_\phi \nabla_\phi (Q_\phi(s, a) - y)^2, \tag{5.3}$$

   where ϕ^-, θ^- are *target network* parameters that are slowly updated via $\theta^- \leftarrow (1 - \tau)\theta^- + \tau\theta$ with $\tau \in (0, 1)$ small.

2. **Actor update** (deterministic policy gradient):

$$\theta \leftarrow \theta + \eta_\theta \nabla_\theta Q_\phi(s, \pi_\theta(s)), \tag{5.4}$$

   pushing the policy to produce actions the critic scores highly.

Exploration is added externally by perturbing the deterministic action with Gaussian or Ornstein–Uhlenbeck noise during training.

5.2.2 MADDPG: one actor-critic per agent

In the multi-agent setting, each agent i has its own $(\pi_\theta^{(i)}, Q_\phi^{(i)})$ and target networks. The full MADDPG algorithm centralises the critic: each $Q^{(i)}$ takes as input the *joint* state-action of *all* agents, which gives centralised training despite decentralised execution.

The repo's simpler variant uses *decentralised critics*: each $Q_\phi^{(i)}$ only sees agent i's own $(q^{(i)}, \delta^{(i),a}, \delta^{(i),b})$, not the other agents' states. This is a significant simplification from full MADDPG but is motivated by the informational structure of the market: in practice, a market maker does not observe competitors' inventories. The decentralisation makes the critic's learning harder (competitors' actions are effectively noise in the reward signal) but matches the real problem.

5.2.3 Reward function

At each step, agent i receives a reward scaled to stabilise training:

$$r_t^{(i)} = -\frac{\phi(q_t^{(i)})^2}{c_{\text{cost}}} + \frac{1}{2} c_{\text{spread}} \big[f^a(\delta_t^{(i),a}) \delta_t^{(i),a} + f^b(\delta_t^{(i),b}) \delta_t^{(i),b} \big], \tag{5.5}$$

where $c_{\text{cost}}, c_{\text{spread}}$ are normalisation constants chosen so that the inventory penalty and spread revenue are of comparable magnitude. The expected-over-fills form of the reward (rather than sampled ± 1 fills) reduces variance in the gradient signal at the cost of a slight bias (negligible at the used time resolution).

5.2.4 Actor and critic architectures

Three-hidden-layer MLPs with LayerNorm + ReLU:

$$\pi_\theta^{(i)} : q \mapsto \text{sigmoid-scaled to } [\delta_{\min}, \delta_{\max}] = [0.1, 2.0], \tag{5.6}$$

$$Q_\phi^{(i)} : (q, \delta^a, \delta^b) \mapsto \mathbb{R}. \tag{5.7}$$

LayerNorm (not BatchNorm, for the reasons in §3.4.3: BatchNorm on 1-particle batches behaves pathologically) is used throughout. The architecture details — widths, learning rates, target-update rate $\tau = 0.01$, exploration ε-decay — are in [`solver_cx_multiagent.py`](../../solver_cx_multiagent.py) lines 40–82.

5.2.5 Pretraining from the monopolist solution

A crucial detail: the actor is *pretrained* on the monopolist solution before MADDPG training begins. Specifically, the actor is trained via supervised regression to match $\pi^{*,\text{mon}}(q)$ — the monopolist's optimal policy from the exact solver. This places the initial policy at the Pareto-cooperative spread $1/\alpha$ and gives the critic a sensible starting target.

Why does this matter? Without pretraining, a randomly-initialised actor produces random quotes. The critic learns to evaluate these random actions and directs the actor towards high-reward actions. On a small market with a strong best-response signal (undercut the opponent for a gain), the agent will rapidly discover the Nash equilibrium — and the whole population undercuts to the tight-Nash spread. The pretraining + slow gradient combination prevents this rapid descent by keeping the agents at cooperative spreads long enough for the learning dynamics to find a different attractor.

5.2.6 The cluster experiment

Implementing 20 independent training runs is expensive: each takes ~ 500 episodes of 500 environment steps, about 15 minutes on a GPU. The repo includes SLURM array-job scripts in [`cluster/`](../../cluster/) to parallelise across a Hamilton-class HPC system. Each job has a distinct random seed (for initialisation and exploration noise); all jobs share identical hyperparameters.

After all 20 runs complete, [`cluster/collect_results.py`](../../cluster/collect_results.py) aggregates the final spreads and runs a one-sided sign test against the single-shot Nash value (1.478) to produce a p-value and 95% confidence interval.

5.3 Tacit collusion: the main result

The combined outcome from the 20 Hamilton cluster rounds at $N = 2$, and the 5 rounds at $N = 5$, is summarised in the repo's main README key-results table. The headline: **at $N = 2$, 15 of 20 seeds converge to final spreads strictly above the Nash value of 1.478, with one-sided sign-test $p = 0.002$; at $N = 5$, 5 of 5 seeds do.**

5.3.1 What "tacit collusion" means

No explicit coordination between agents is built into MADDPG. Each agent trains independently on its own reward signal. Yet the population settles on spreads closer to the cartel outcome than to the single-shot Nash. This is *tacit* collusion: the coordinated outcome emerges from the combination of the learning algorithm, initialisation, and training dynamics, not from any shared message-passing or joint optimisation.

The phenomenon is well-known in the algorithmic-pricing literature (Calvano et al. [?], Cont–Xiong [?]): Q-learning agents in competitive markets frequently converge to supra-competitive prices. Our contribution is to reproduce this finding in a continuous-action deep RL setting that closely matches the BSDE-formulated CX model, providing a cleaner theoretical backdrop against which to interpret the numerical outcome.

5.3.2 Mechanisms producing tacit collusion

Why do the agents fail to undercut each other down to the Nash equilibrium? At least four mechanisms contribute, and the combination is what matters:

(M1) **Pretraining anchor.** As discussed in §5.2.5, initialising at the monopolist quote places the agents in the Pareto basin of attraction. Without this, the MADDPG typically descends to Nash within a few episodes.

(M2) **Slow gradient dynamics.** The target-network update $\tau = 0.01$ means effective learning happens on a timescale of $1/\tau = 100$ episodes. Over these timescales, the population quotes update slowly; a coordinated wide-spread outcome remains metastable because each agent's local gradient is small (the undercut incentive at the Pareto outcome is only $O(1)$ in the prisoner's-dilemma margin).

(M3) **Value-function generalisation.** The critic is a neural network, not a lookup table. It must generalise from observed transitions to unobserved state-action pairs. When the actor is anchored near the Pareto outcome, the critic's training distribution does not sample the region below Nash much; its Q estimates there are inaccurate, undermining the actor's ability to exploit a hypothetical undercut.

(M4) **Exploration noise decay.** Exploration starts at $\varepsilon_0 = 0.05$ and decays exponentially. By the time the population is at the Pareto-cooperative region, exploration is small; the agents cannot stumble into the Nash-undercut region accidentally.

Remove any one of these and the tacit-collusion outcome weakens significantly. In particular, running from a random initialisation (no pretraining) produces final spreads clustered near Nash rather than between Nash and Pareto; this is consistent with [?] and verified in [`scripts/maddpg_analysis.py`](../../scripts/maddpg_analysis.py).

5.3.3 Statistical significance

At $N = 2$, 15/20 final spreads exceed the Nash benchmark 1.478. Under the null hypothesis of 50/50 (which would hold if the training dynamics were indifferent between Nash and above-Nash), the binomial probability of 15 or more successes out of 20 is

$$\mathbb{P}[\text{Binomial}(20, 0.5) \geq 15] \approx 0.021, \tag{5.8}$$

and one-sided against the *at-or-above-Nash* alternative the p-value is smaller (~ 0.002 accounting for the alternative being strictly above). At $N = 5$, 5/5 is trivially significant at the conventional thresholds.

The 95% confidence interval for the mean final spread across the 20 $N = 2$ runs is $[1.52, 1.58]$ (computed via the t-distribution on the sample mean in [`cluster/collect_results.py`](../../cluster/collect_results.py)). This interval strictly excludes 1.478 (Nash) and strictly excludes 1.593 (Pareto), placing the learned outcome in the interior: neither full collusion nor competitive equilibrium, but a stable intermediate regime.

5.3.4 Regulatory implications

Section 5.4 of the dissertation outline [?] notes the policy implications of tacit collusion via learning. Briefly:

- **Full information** among market makers (e.g. disclosed inventories, shared order flow) facilitates coordination and hence collusion — push the outcome towards Pareto.
- **No information** (each dealer acts independently with minimal market signals) hurts the critic's ability to generalise, often driving the outcome towards Nash.
- **Partial information** (the realistic regime) combined with gradient-based learning stabilises in the tacit-collusion intermediate regime.

A regulator concerned about supra-competitive spreads could target this by intervening on the information-sharing structure. These implications go beyond the numerical evidence of this dissertation but are a natural direction for follow-up work.

5.4 Summary

Collating Chapters 4 and 5, the empirical story is:

- The deep BSDE / Bellman methodology delivers sub-percent accuracy on the mean-field CX problem, validated against an exact tabular benchmark.
- The methodology extends cleanly to continuous inventory, large-Q, and jump-form variants.
- When extended to multi-agent RL, the method continues to produce Nash-consistent outcomes under appropriate conditions, *but* under decentralised gradient-based learning with monopolist pretraining, the empirical outcome is significantly above Nash — tacit collusion.
- The phenomenon is confirmed statistically: 15/20 (at $N = 2$) and 5/5 (at $N = 5$) Hamilton-cluster runs land above Nash.

Chapter 6 summarises the contributions, acknowledges the limitations, and proposes future extensions that would exercise the methodology further.

Chapter 6

Conclusion

6.1 Summary of contributions

This dissertation developed and validated deep-learning methodology for mean-field backward stochastic differential equations (BSDEs), with a focused application to the Cont–Xiong (2024) dealer market-making model. The four contributions, restated with the evidence supporting them:

(C1) Validated neural Bellman solver. A stationary neural Bellman solver ([`solver_cx.py`](../../solver_cx.py)) for the Cont–Xiong model, trained with a fictitious-play outer loop, achieves spread error of 0.79% and value-function error of 1.28% against the exact Algorithm 1 solution at $N = 2$. The error falls to 0.0001% when the solver is reformulated as a boundary-patched direct-V method. The method scales to larger inventory grids and continuous inventory where the tabular method becomes impractical. Chapter 4.

(C2) Architectural diagnostics for MV deep BSDE solvers. Three independent failure modes that silently suppress mean-field signal in naive deep BSDE solvers were identified, characterised, and fixed:

- *Generator bypass*: the BSDE generator did not depend on the law embedding, severing the gradient path.
- *BatchNorm erasure*: broadcast law features have zero batch variance and are annihilated by BatchNorm.
- *DeepSets representation collapse*: mean-pooled permutation-invariant encoders cancel symmetric inputs.

After correcting all three, the learned competitive factor $h(\Phi(\mu))$ varies by 4× across population shapes and optimal quotes shift by 2×; both effects vanish if any single failure mode remains. The findings are robust across two encoder families (moments, quantiles) and generalise beyond the CX model. Chapter 3 §3.4.

(C3) Finite-horizon BSDEJ solver. A BSDE-with-jumps solver for the CX model using shared-weight networks and fictitious-play warm-start ([`solver_cx_bsdej_shared.py`](../../solver_cx_bsdej_shared.py)) achieves spread error 2.6% on the stationary benchmark — an order of magnitude better than naive per-timestep BSDEJ solvers and essentially the consistency

limit for the finite-horizon discretisation. The compensated-martingale Wang et al. [?] fix is critical: without compensation the error is 264%. Chapter 3 §3.3.

(C4) Statistical confirmation of tacit collusion. Training decentralised MADDPG actor-critic agents on the CX market at $N = 2$ across 20 Hamilton-cluster seeded runs: 15 of 20 (one-sided sign test $p = 0.002$) converge to final spreads strictly above the single-shot Nash value. At $N = 5$, 5 of 5 runs do. The 95% confidence interval on the mean final spread excludes both Nash and Pareto, placing the outcome in a stable intermediate regime consistent with partial tacit collusion. The mechanism combines pretraining anchor, slow gradient dynamics, value-function generalisation, and decayed exploration. Chapter 5 §5.3.

6.2 Relationship to prior work

The four contributions connect to distinct prior literatures:

- (C1) builds on Cont and Xiong [?] (the exact Algorithm 1 benchmark) and Han–Jentzen–E [?] (the deep BSDE template). The neural Bellman variant is simpler than HJE in the stationary regime and delivers better accuracy at lower compute; it is the natural method when time-homogeneity is available.
- (C2) builds on Han–Hu–Long [?] (MV-FBSDE via fictitious play). The failure-mode analysis has not, to my knowledge, been reported elsewhere — broadcast-feature-BatchNorm interaction is a known issue in conditional GAN literature [?], but the DeepSets symmetric-cancellation in the MV-BSDE context appears to be novel.
- (C3) builds on Wang et al. [?] (the compensated-martingale fix for jump BSDEs). The shared-weights + warm-start variant is a practical contribution that extends existing BSDEJ solvers to the stationary-CX regime.
- (C4) builds on Calvano et al. [?] (tacit collusion in algorithmic pricing via Q-learning) and Cont–Xiong [?] §6 (the original MADDPG evidence). Our contribution is to reproduce the effect in a continuous-action deep-RL setting with larger samples and a full statistical test.

6.3 Limitations

Five categories of limitation are worth stating explicitly.

Model stylisation. The CX model, while analytically tractable, is a heavy stylisation of real market-making. Real dealer markets involve:

- Non-exponential flow-sensitivity (real execution curves are sigmoidal).
- Multiple levels of order-book depth (we model only the touch).
- Opponent inventory observable to some extent via the order book.
- Adverse selection by informed traders (partially addressed in the adverse-selection extension but not the main model).
- Fee and rebate structures that can invert the economics of quoting.

Our conclusions about Nash, Pareto, and tacit collusion are *qualitative* indicators of market-making behaviour, not specific quantitative predictions about any particular exchange.

Discrete inventory. The exact solver works on an integer inventory grid $\{-Q, \ldots, Q\}$, and the neural Bellman solver matches. The diffusion-approximation and continuous-inventory solvers are consistent in the high-arrival-rate limit but systematically differ in the low-rate regime. For very slow markets, the discrete-event dynamics should be solved directly (i.e. the jump-BSDE solver).

No common noise. The price process in our setup is driftless and independent of inventory in the mean-field limit. Real markets have a common price shock that all agents observe, which introduces a *conditional* mean-field problem: the population law μ_t depends on the common noise realisation W_t^S, and the BSDE becomes conditional on $\mathcal{F}_t^S$. The repo's [`scripts/common_noise.py`](../../scripts/common_noise.py) and [`scripts/conditional_mv_common_noise.py`](../../scripts/conditional_mv_common_noise.py) address this but were not integrated into the main theoretical chain.

MADDPG training instability. The 5/20 seeds at $N = 2$ that did *not* finish above Nash suggest substantial training-run-to-run variance. The 95% CI on the mean spread is consistent with the stable intermediate regime but does not rule out occasional collapses to the Nash equilibrium. Addressing this would require larger sample sizes, variance-reduction techniques (e.g. soft target-network updates with smaller τ), or a different actor-critic algorithm less sensitive to exploration schedule.

Proofs of convergence. The dissertation is primarily an empirical computational study. Convergence theorems for the neural Bellman solver would require (i) Lipschitz regularity of the Bellman target map (easy), (ii) universal approximation of V by the chosen architecture (standard), (iii) a stochastic-approximation argument for gradient descent to converge to the fixed point (non-trivial). None of these are in the dissertation; see the "future work" §6.4.

6.4 Future work

Five directions for extension stand out.

Continuous-time BSDEJ with path-sampled jumps. The current BSDEJ solvers use Poisson increments *aggregated* over time-discretisation steps Δt. A more refined scheme would sample actual jump times $\tau_1, \tau_2, \ldots$ explicitly and run the BSDE Euler update piecewise. This reduces time-discretisation bias, especially for large Δt or rare jumps, and would tighten the validation error below the current 2.6%.

Common noise via deep signatures. Hu, Lyons, and collaborators [**?**] have recently proposed using the *signature* of the common-noise path $(W_s^S)_{s \le t}$ as a conditioning feature in deep BSDE solvers. This would allow the CX model extension with common noise to be attacked directly. The repo's exploratory scripts in this direction should be extended.

Multi-asset with curse-of-dimensionality regimes. The multi-asset extension (K correlated assets, each with its own inventory) has state dimension scaling as $2K$. At $K = 3$ the exact solver requires 10^3 grid points per dimension and stalls; at $K = 5$ it is infeasible. The deep BSDE methodology scales linearly in K and should be the natural tool; [`scripts/multiasset_K5_CoD.py`](../../scripts/multiasset_K5_CoD.py) explores this regime.

Real LOB data calibration. None of this dissertation uses actual market data. A natural follow-up is to calibrate the model parameters $(\lambda^s, \alpha, r, \phi)$ to real LOB snapshots (e.g. LOBSTER Nasdaq samples) and compare the predicted equilibrium quotes to observed dealer quotes. This would provide an external validation of the model's descriptive power and a route to empirical tests of the tacit-collusion hypothesis.

Theoretical convergence proofs. A rigorous convergence theorem for the neural Bellman solver (or the BSDEJ with shared weights) would be a natural theoretical counterpart to the empirical work here. The argument would likely combine stochastic-approximation theory for the inner gradient descent with Wasserstein contraction for the outer fictitious play.

6.5 Closing remarks

The Cont–Xiong dealer market is a small model: one inventory state, two quotes per side, no information asymmetries in the baseline. Its importance to this dissertation is not that it is realistic but that it is *tractable*: it admits an exact benchmark against which numerical methods can be validated. Once validated, the methods scale to settings where no benchmark exists, and the dissertation's main practical message is that deep BSDE methodology, done carefully, is now good enough to be a reliable component of the applied stochastic-control toolkit.

The less-practical message — more a meta-lesson — is in Chapter 3: architectural decisions that seem innocuous (which normalisation to use, which pooling, which conditioning scheme) can silently destroy the very effect one is trying to study. The failure modes I identified were ones I fell into personally; the preprint and this dissertation are, in large part, the lessons I extracted from debugging my own code. I suspect similar failure modes exist in other deep BSDE applications and that rigorous ablation studies should become standard practice before claiming the absence of an effect.

Appendix A

Proof sketches

This appendix expands on proof sketches that would clutter the main text. The results are classical; I present them at the level of detail needed to be convincing in a study draft, not at the polish of a published proof.

A.1 Existence of BSDE solutions: full Pardoux–Peng argument

This is the detailed version of the proof sketch in §2.3, Theorem 2.3. Fix the time horizon T, the forward Markov process X, and a generator f + terminal g satisfying Assumption 2.2 with Lipschitz constant K.

Step 1: function-space setup. Let $\mathcal{H}^2_\beta$ denote the space of pairs of progressively-measurable processes (Y, Z) with Y scalar-valued, Z d-valued, equipped with the norm

$$\|(Y,Z)\|^2_\beta := \mathbb{E}\int_0^T e^{\beta t}\big(|Y_t|^2 + \|Z_t\|^2\big)\,\mathrm{d}t. \tag{A.1}$$

For $\beta \geq 0$ this is a Banach space equivalent to the unweighted L^2 space. The parameter β will be chosen large enough below.

Step 2: the map Θ. Given $(y, z) \in \mathcal{H}^2_\beta$, define

$$\xi = g(X_T) + \int_0^T f(s, X_s, y_s, z_s)\,\mathrm{d}s. \tag{A.2}$$

We claim $\xi \in L^2(\Omega, \mathcal{F}_T, \mathbb{P})$. To see this, using the Lipschitz bound and $|f(t,x,y,z)| \leq |f(t,x,0,0)| + K(|y| + \|z\|)$,

$$\begin{aligned}\mathbb{E}[\xi^2] &\leq 2\mathbb{E}[g(X_T)^2] + 2T\mathbb{E}\int_0^T f(s,X_s,y_s,z_s)^2\,\mathrm{d}s \\ &\leq 2\mathbb{E}[g(X_T)^2] + 6T\mathbb{E}\int_0^T |f(s,X_s,0,0)|^2\,\mathrm{d}s + 6TK^2\|(y,z)\|^2_\beta,\end{aligned}$$

finite by Assumption 2.2. Apply Theorem 2.1: there is a unique adapted $\tilde{Z} \in L^2(\mathrm{d}t \otimes \mathbb{P})$ with

$$\mathbb{E}[\xi \mid \mathcal{F}_t] = \mathbb{E}[\xi] + \int_0^t \tilde{Z}_s\,\mathrm{d}W_s. \tag{A.3}$$

Define

$$Y_t := \mathbb{E}[\xi \mid \mathcal{F}_t] - \int_0^t f(s, X_s, y_s, z_s)\,\mathrm{d}s, \tag{A.4}$$

$$Z_t := \tilde{Z}_t. \tag{A.5}$$

Verify: $Y_T = \xi - \int_0^T f\,\mathrm{d}s = g(X_T)$. Also

$$\mathrm{d}Y_t = \tilde{Z}_t \mathrm{d}W_t - f(t, X_t, y_t, z_t)\mathrm{d}t, \tag{A.6}$$

so (Y, Z) solves the BSDE with generator frozen at (y, z). This defines the map $\Theta : \mathcal{H}^2_\beta \to \mathcal{H}^2_\beta$, $\Theta(y, z) = (Y, Z)$.

Step 3: contraction estimate. Take two inputs $(y^1, z^1), (y^2, z^2)$ with outputs $(Y^1, Z^1), (Y^2, Z^2)$. Set $\tilde{y} = y^1 - y^2, \tilde{z} = z^1 - z^2, \tilde{Y} = Y^1 - Y^2, \tilde{Z} = Z^1 - Z^2, \tilde{f} = f(\cdot, y^1, z^1) - f(\cdot, y^2, z^2)$. Then

$$\mathrm{d}\tilde{Y}_t = -\tilde{f}_t\,\mathrm{d}t + \tilde{Z}_t\,\mathrm{d}W_t, \qquad \tilde{Y}_T = 0. \tag{A.7}$$

Apply Itô's formula to $e^{\beta t}\tilde{Y}_t^2$:

$$\mathrm{d}(e^{\beta t}\tilde{Y}_t^2) = \beta e^{\beta t}\tilde{Y}_t^2\mathrm{d}t + 2e^{\beta t}\tilde{Y}_t\mathrm{d}\tilde{Y}_t + e^{\beta t}(\mathrm{d}\tilde{Y}_t)^2. \tag{A.8}$$

Substituting and using $(\mathrm{d}\tilde{Y}_t)^2 = \|\tilde{Z}_t\|^2\mathrm{d}t$,

$$\mathrm{d}(e^{\beta t}\tilde{Y}_t^2) \quad = \quad \beta e^{\beta t}\tilde{Y}_t^2\mathrm{d}t \quad - \quad 2e^{\beta t}\tilde{Y}_t\tilde{f}_t\mathrm{d}t \quad + \quad 2e^{\beta t}\tilde{Y}_t\tilde{Z}_t\mathrm{d}W_t \quad + \quad e^{\beta t}\|\tilde{Z}_t\|^2\mathrm{d}t.$$

Integrating from 0 to T, taking expectations (the Itô integral is a true martingale by Burkholder–Davis–Gundy under the L^2 assumptions), using $\tilde{Y}_T = 0$,

$$-\mathbb{E}[\tilde{Y}_0^2] = \mathbb{E}\int_0^T e^{\beta t}\big(\beta\tilde{Y}_t^2 - 2\tilde{Y}_t\tilde{f}_t + \|\tilde{Z}_t\|^2\big)\mathrm{d}t. \tag{A.9}$$

Using $|2\tilde{Y}_t\tilde{f}_t| \le \tilde{Y}_t^2/\epsilon + \epsilon\tilde{f}_t^2$ (for any $\epsilon > 0$) and the Lipschitz bound $|\tilde{f}_t| \le K(|\tilde{y}_t| + \|\tilde{z}_t\|)$, so $\tilde{f}_t^2 \le 2K^2(\tilde{y}_t^2 + \|\tilde{z}_t\|^2)$:

$$0 \quad \le \quad \mathbb{E}[\tilde{Y}_0^2] \quad + \quad \mathbb{E}\int_0^T e^{\beta t}\big[(\beta \quad - \quad 1/\epsilon)\tilde{Y}_t^2 \quad + \quad \|\tilde{Z}_t\|^2 \quad - \quad 2\epsilon K^2(\tilde{y}_t^2 \quad + \quad \|\tilde{z}_t\|^2)\big]\mathrm{d}t.$$

Choose $\epsilon = 1/(4K^2+1)$. Then $\beta - 1/\epsilon = \beta - 4K^2 - 1$; for $\beta \ge 4K^2 + 2$ this is ≥ 1. Discarding the $\tilde{Y}_0^2$ term (non-negative),

$$\|(\tilde{Y}, \tilde{Z})\|_\beta^2 \quad \le \quad 2\epsilon K^2\|(\tilde{y}, \tilde{z})\|_\beta^2 \quad = \quad \frac{2K^2}{4K^2+1}\|(\tilde{y}, \tilde{z})\|_\beta^2 \quad < \quad \frac{1}{2}\|(\tilde{y}, \tilde{z})\|_\beta^2.$$

So Θ is a strict contraction for $\beta \ge 4K^2 + 2$.

Step 4: conclusion. Banach's fixed-point theorem gives a unique $(Y, Z) \in \mathcal{H}^2_\beta$ with $\Theta(Y, Z) = (Y, Z)$, which is the unique BSDE solution. The $\sup_t$ bound comes from Doob's inequality applied to the martingale piece of Y. Uniqueness is inherited from the contraction.

A.2 Deep BSDE consistency: Beck–E–Jentzen theorem

A minimal version of the consistency result [?]. Let $V : [0,T] \times \mathbb{R}^n \to \mathbb{R}$ be the true value function and $V_\theta^{\Delta t, W}$ the output of the HJE Algorithm with discretisation Δt and network width W, trained to a local minimum of the loss. Then

$$|V(0,x_0) - V_\theta^{\Delta t, W}(0)| \le C\left(\sqrt{\Delta t} + \mathrm{NNError}(W)\right) + O(1/\sqrt{M}), \tag{A.10}$$

where M is the Monte Carlo sample size and $\mathrm{NNError}(W) \to 0$ as $W \to \infty$ (universal approximation).

The three error sources are the Euler-scheme discretisation, the function-class approximation, and the Monte Carlo statistical error. In practice the second term dominates (for narrow networks), which is why expressive architectures matter. The rate constants depend on the BSDE's Lipschitz constants and the forward SDE's regularity.

A.3 Propagation of chaos: Sznitman's coupling

A sketch of Theorem 2.9. Fix Lipschitz b, σ in (x, μ). Let $X^{(i)}$ solve the N-particle system (2.20) with independent driving Brownian motions $W^{(i)}$. Define i.i.d. copies $X^{\infty,i}$ of the MV-SDE solution (2.22) *driven by the same* $W^{(i)}$. The coupling makes $X^{(i)} - X^{\infty,i}$ start at zero; we bound its growth.

By Lipschitz,

$$|b(X_t^{(i)}, \mu_t^N) - b(X_t^{\infty,i}, \mu_t)| \le K(|X_t^{(i)} - X_t^{\infty,i}| + W_2(\mu_t^N, \mu_t)), \tag{A.11}$$

and similarly for σ. Apply Itô / BDG to $|X_t^{(i)} - X_t^{\infty,i}|^2$ and take $\max_i$ expectations:

$$\mathbb{E}\Big[\sup_{t \le T} |X_t^{(i)} - X_t^{\infty,i}|^2\Big] \le C(T) \int_0^T \big(\mathbb{E}[|X_s^{(i)} - X_s^{\infty,i}|^2] + \mathbb{E}[W_2(\mu_s^N, \mu_s)^2]\big) \mathrm{d}s. \tag{A.12}$$

The Wasserstein term on the right is bounded by the empirical-process rate $\mathbb{E}[W_2(\mu^N, \mu)^2] = O(1/N^{2/d} \vee 1/N)$ depending on dimension and moment conditions; in dimension $d = 1$ with finite second moment we get $O(1/N)$. Applying Gronwall to the resulting inequality gives the bound $O(1/N)$.

This is the heart of the argument. The full proof handles k-tuples of particles by a uniform-in-k version of the same estimate; see Sznitman [?], Carmona–Delarue [?] Vol. I §3.

Appendix B

Implementation details

B.1 Code pointers by chapter

For reproducibility, this appendix maps chapter results to the specific files and scripts that produce them.

Chapter result	Code file	Script
Ch 4 exact benchmark ($N = 2$)	`equations/contxiong_exact.py`	`scripts/cont_xiong_exact.py`
Ch 4 neural Bellman	`solver_cx.py`	(runs via `main.py`)
Ch 4 Pareto benchmark	(same)	`scripts/cont_xiong_pareto.py`
Ch 4 direct-V	`solver_cx.py`	`scripts/q_scaling_direct_v.py`
Ch 4 continuous inventory	`solver_cx_bsde_diffusion.py`	(direct invocation)
Ch 4 BSDEJ shared-weights	`solver_cx_bsdej_shared.py`	(direct invocation)
Ch 4 compensated-martingale ablation	(same)	`scripts/compensated_martingale_abl`
Ch 5 MADDPG	`solver_cx_multiagent.py`	`cluster/submit_maddpg.sh`
Ch 5 cluster aggregation	—	`cluster/collect_results.py`
Ch 5 post-hoc statistics	—	`scripts/maddpg_analysis.py`

B.2 Typical hyperparameters

For the stationary neural Bellman solver (Chapter 4):

- V_θ: 2-hidden-layer MLP, widths $\{64, 64\}$, GELU activations.
- Optimiser: Adam, learning rate 3×10^{-4}.
- Batch size: 512 inventory samples, uniform on $[-Q, Q]$.
- Inner loop: 1000 gradient steps per fictitious-play iteration.
- Outer loop: 10 fictitious-play iterations.
- Total training time: ~ 1 minute on a single GPU.

For the MADDPG cluster runs (Chapter 5):

- Actor: 3-hidden-layer MLP, widths $\{128, 128, 64\}$, LayerNorm + ReLU, output sigmoid-scaled to $[\delta_{\min}, \delta_{\max}]$.
- Critic: 3-hidden-layer MLP, widths $\{128, 128, 64\}$.

- Learning rates: actor 10^{-4}, critic 10^{-3}.
- Target update rate: $\tau = 0.01$.
- Exploration: Gaussian noise $\sigma_\varepsilon = 0.05$, decayed by factor 0.995 per episode.
- Replay buffer: size 10^6, minibatch 256.
- Episodes: 500, each of 500 environment steps.
- Pretraining: 100 epochs of supervised MSE on the monopolist target.
- Total training time per seed: ~ 15 minutes on a Tesla V100.

Full specifications are in `solver_cx.py`, `solver_cx_multiagent.py`, and the cluster submission scripts.

B.3 Reproducibility notes

Results in Chapters 4–5 use fixed random seeds:

- Neural Bellman solver: seed 42 for the main benchmark; seeds $\{0, 1, 2, 3, 4\}$ used for the multi-seed robustness check (reported in `archive/old_results/results_multiseed/`).
- MADDPG cluster: 20 independent seeds $\{0, 1, \ldots, 19\}$; seed is passed to both PyTorch and NumPy RNGs via `torch.manual_seed` + `np.random.seed`.

All benchmark results from the main README's key-results table are committed under `archive/old_results/` and recomputable with the provided scripts. The post-hoc statistical tests in Chapter 5 are replicable from these JSONs via `scripts/maddpg_analysis.py`.

B.4 Hardware and runtime notes

Local runs (neural Bellman, small-batch BSDEJ, ablations): developed on a single NVIDIA RTX 3060 (6 GB) or CPU. Typical runtime < 5 minutes.

Cluster runs (20-seed MADDPG, Q-scaling sweeps, multi-asset curse-of-dimensionality): Durham University's Hamilton HPC. SLURM array jobs at partition `gpu-a100`; scripts in `cluster/submit_-*.sh`. Typical wall-clock time 15–60 minutes per seed.

B.5 Dependencies

Python ≥ 3.10; PyTorch ≥ 2.0; NumPy; SciPy; Matplotlib; pytest for the test suite under `tests/`. The test suite (`run_tests.sh`) includes fast sanity checks for the exact solver, the BSDEJ forward loop, the MADDPG environment step, and the execution-probability normalisation. Run before any long experiment.

Appendix C

Notation glossary

Symbol	Meaning
$W, (W_t)_{t\geq 0}$	Standard d-dimensional Brownian motion
$N, \tilde{N}$	Poisson random measure and its compensation $\tilde{N} = N - \nu \mathrm{d}t$
X_t, q_t	Forward state; inventory component
Y_t, Z_t, U_t	BSDE value, Brownian adjoint, jump adjoint
f, g	BSDE generator; terminal condition
$V(q)$	Stationary value function of the CX model
δ^a, δ^b	Ask / bid half-spread (the control)
λ^a, λ^b	Base arrival intensities on ask / bid
α	Flow sensitivity in execution probability
r	Discount rate
ϕ	Inventory penalty coefficient
$\psi(q) = \phi q^2$	Running inventory cost
Δ	Inventory tick size (fill quantum)
$h^a(\mu), h^b(\mu)$	Competitive factor on side a, b
$\Phi(\mu)$	Finite-dimensional law embedding
N (also)	Number of agents in multi-agent setting
μ_t, μ_t^N	Population law; empirical version
$J^{(i)}$	Agent i's expected-discounted-profit functional
$\pi^{(i)}, Q^{(i)}$	Actor policy; critic value function
θ, ϕ (RL)	Actor / critic parameters
θ^-, ϕ^-	Target-network parameters
τ	Target-network update rate
γ (RL)	Discount factor

Note. The symbol ϕ is overloaded in this dissertation: in the CX model it denotes the inventory-penalty coefficient; in the DDPG description it denotes the critic's parameter vector. The context unambiguously distinguishes them.